

A Major Project Report (Stage – II)
On
**“A Time-Frequency Based Suspicious Activity Detection
for Anti-Money Laundering”**

**Submitted in Partial Fulfillment of the Academic
Requirement for the Award of Degree of**

BACHELOR OF TECHNOLOGY
In
Computer Science and Engineering (Data Science)

Submitted By:

R. HEMANTH (22R01A67B4)

Under the esteemed guidance of

Mr. P. Rajendran
(Assistant Professor, CSE - Data Science)



CMR INSTITUTE OF TECHNOLOGY
(UGC AUTONOMOUS)

Approved by AICTE, Affiliation to JNTUH, Accredited by NAAC with ‘A+’ Grade

Kandlakoya(V), Medchal Dist-501 401

www.cmrithyderabad.edu.in

2025-26

CMR INSTITUTE OF TECHNOLOGY

(UGC AUTONOMOUS)

Approved by AICTE, Affiliated to JNTUH, Accredited by NAAC with 'A+' Grade

Kandlakoya(V), Medchal Dist-501 401

www.cmrihyderabad.edu.in



CERTIFICATE

This is to certify that an Industry oriented Major Project (Stage – II) entitled with “A Time-Frequency Based Suspicious Activity Detection for Anti-Money Laundering” is being submitted by:

R.HEMANTH

(22R01A67B4)

To JNTUH, Hyderabad, in partial fulfillment of the requirement for award of the degree of B.Tech in CSE (Data Science) and is a record of a bonafide work carried out under our guidance and supervision. The results in this project have been verified and are found to be satisfactory. The results embodied in this work have not been submitted to have any other University for award of any other degree or diploma.

Signature of Guide

Mr.P. Rajendran

Signature of Project Coordinator

Mr.P. Rajendran

Signature of HOD

Dr.A. Nirmal Kumar

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We are extremely grateful to **Dr.M. Janga Reddy**, Director, **Dr.B. Satyanarayana**, Principal and **Dr.A.Nirmal Kumar**, Head of Department, Department of Computer Science and Engineering (Data Science), CMR Institute of Technology for their inspiration and valuable guidance during entire duration.

We are extremely thankful to **Mr.P. Rajendran**, Major Project Coordinator and internal guide Dept of Computer Science and Engineering (Data Science), CMR Institute of Technology for their constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if we do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We express our thanks to all staff members and friends for all the help and coordination extended in bringing out this Project successfully in time.

Finally, we are very much thankful to our parents and relatives who guided directly or indirectly for every step towards success.

R.HEMANTH

(22R01A67B4)

ABSTRACT

Money laundering is the crucial mechanism utilized by criminals to inject proceeds of crime into the financial system. The primary responsibility of the detection of suspicious activity related to money laundering is with the financial institutions. Most of the current systems in these institutions are rule-based and ineffective (over 90 % false positives). The available data science based anti- money laundering (AML) models to replace the existing rule-based systems work on customer relationship management (CRM) features and time characteristics of transaction behavior. Due to thousands of possible account features, customer features, and their combinations, it is challenging to perform feature engineering to achieve reasonable accuracy.

Aiming to improve the detection performance of suspicious transaction monitoring systems for AML systems, in this article, we introduce a novel feature set based on time-frequency analysis that uses 2-D representations of financial transactions. Random forest is utilized as a machine learning method, and simulated annealing is adopted for hyperparameter tuning. The designed algorithm is tested on real banking data, proving the results' efficacy in practically relevant environments. It is shown that the time-frequency characteristics are discriminatory features for suspicious and non-suspicious entities. Therefore, these features substantially improve the area under curve results (over 1%) of the existing data science-based transaction monitoring systems. Using time-frequency features alone, a false positive rate of 14.9% has been achieved, with an Fscore of 59.05%. When combined with transaction and CRM features, the false positive rate is 11.85%, and the F-Score is improved to 74.06%

Introduction

INDEX

S.No	Chapter / Section	Page No.
1.	Introduction	1
2.	Objectives of the project	2
3.	Literature Survey	3
	3.1. Existing System	5
	3.2. Limitations of Existing System	5
	3.3. Proposed System	5
	3.4. Advantages of Proposed System	5
4.	Requirement Specification	13
5.	Proposed Methodology	16
6.	System Design	19
	6.1. Architecture	19
	6.2. UML Diagram	21
	1. Class Diagram	23
	2. UseCase Diagram	24
	3. Sequence Diagram	25
	4. Collaboration Diagram	26
	5. Component Diagram	27
	6. Deployment Diagram	28
	7. Activity Diagram	28
	8. Data Flow Diagram	29
7.	Implementation	30
	7.1. Python	30
	7.2. Code Snippets	32
8.	Testing	37
9.	Results	40
10.	Conclusion	46
11.	Future Enhancement	47
12.	Reference	48

List of Figures

Figure Number	Description	Page No.
Fig 1	SDLC (Umbrella Model)	6
Fig 2	Requirements Gathering	7
Fig 3	Analysis Stage	8
Fig 4	Designing Stage	9
Fig 5	Development Stage	10
Fig 6	Integration & Test Stage	11
Fig 7	Installation & Acceptance Test	12
Fig 8	Architecture	19
Fig 9	Class Diagram	23
Fig 10	Use Case Diagram	24
Fig 11	Sequence Diagram	25
Fig 12	Collaboration Diagram	26
Fig 13	Component Diagram	27
Fig 14	Deployment Diagram	28
Fig 15	Activity Diagram	28
Fig 16	Data Flow Diagram	29
Fig 17	Transaction Data Features	40
Fig 18	User Interface	41
Fig 19	Upload AML Dataset	41
Fig 20	Graph	42
Fig 21	Processed Dataset Values	42
Fig 22	Random Forest F-Score on Transaction Data	43
Fig 23	Random Forest F-Score on Transaction & Time Frequency Data	44
Fig 24	Comparison Graph	44
Fig 25	Test Dataset	45
Fig 26	Prediction	45

1. INTRODUCTION

Money laundering (ML) is the umbrella under which the legitimization of the proceeds of crime is attempted while laundered money can be both re-inserted into the legitimate economy and reused to fuel further criminal activities. All major criminality such as drug and human trafficking, terrorism, extortion, kidnap-for-ransom, bribery, embezzlement, tax evasion, corruption and a multiplicity of other offenses (also known as predicate offenses) are connected through ML.

Even though it is impossible to provide an accurate estimate of the size of such a complex underground market, the International Monetary Fund (IMF) indicates that every year, up to 2 trillion USD is laundered through financial systems globally, making ML one of the world's largest markets.

To tackle this issue, most countries following the Financial Action Task Force (FATF) recommendations set up an anti-money laundering (AML) structure, as shown in Fig.1. It is the responsibility of the financial institutions to report suspicious activities to the Financial Intelligence Unit (FIU). The FIU collects intelligence from all different financial institutions within and outside the jurisdiction, which are later reported to the law enforcement agencies (LEA) as necessary. The police, using this intelligence, builds a case to the judicial system, and if ordered the Asset Recovery Bureau (ARB), recovers the suspicious assets for the public, closing the loop. As the initiator of the whole process, identifying the suspicious activity by the financial institutions is very critical.

While technology is essential for the processing and identification of suspicious transactions given the volume of data that needs to be filtered, technology adoption in an AML- context needs to be carefully balanced against the various stakeholders in the AML chain of investigation. Also, most of the existing proposed software' solutions' are rule based, and only 25% of respondents have already implemented Artificial Intelligence (AI) and expressed their main business drivers for machine learning in AML as anomaly detection, segmentation, and model tuning . However, the use of AI instead of rule-based engines for new cases detection is infrequent.

2. OBJECTIVES OF THE PROJECT

Money laundering is the crucial mechanism utilized by criminals to inject proceeds of crime into the financial system. The primary responsibility of the detection of suspicious activity related to money laundering is with the financial institutions. Most of the current systems in these institutions are rule-based and ineffective (over 90 % false positives). The available data science based anti- money laundering (AML) models to replace the existing rule-based systems work on customer relationship management (CRM) features and time characteristics of transaction behavior. Due to thousands of possible account features, customer features, and their combinations, it is challenging to perform feature engineering to achieve reasonable accuracy. Aiming to improve the detection performance of suspicious transaction monitoring systems for AML systems, in this article, we introduce a novel feature set based on time-frequency analysis that uses 2-D representations of financial transactions. Random forest is utilized as a machine learning method, and simulated annealing is adopted for hyperparameter tuning.

The designed algorithm is tested on real banking data, proving the results' efficacy in practically relevant environments. It is shown that the time-frequency characteristics are discriminatory features for suspicious and non- suspicious entities. Therefore, these features substantially improve the area under curve results (over 1%) of the existing data science-based transaction monitoring systems. Using time-frequency features alone, a false positive rate of 14.9% has been achieved, with an Fscore of 59.05%. When combined with transaction and CRM features, the false positive rate is 11.85%, and the F-Score is improved to 74.06%. The main objective of this project is to develop a clear understanding of money laundering, its meaning, and the various stages through which illegally obtained money is transformed into seemingly legitimate funds. The project aims to identify and study the different methods and techniques used by criminals, such as smurfing, shell companies, trade manipulation, and the growing use of digital platforms and cryptocurrencies. Another important objective is to analyze the economic and social impact of money laundering, including how it weakens financial systems, promotes corruption, reduces government revenue, and supports other criminal activities.

3. LITERATURE SURVEY

[1] United Nations Office on Drugs and Crime (UNODC), "**Estimating Illicit Financial Flows Resulting From Drug Trafficking and Other Transnational Organized Crimes**," UNODC, Vienna, Austria, 2011. This study estimates that approximately 2.7% of global GDP — around U.S. \$1.6 trillion in 2009 — represents criminally obtained money laundered by transnational organized crime. Illicit drug sales account for 20% of all crime proceeds, with global cocaine gross profits estimated at U.S. \$84 billion, predominantly generated in North America (U.S. \$35 billion) and West and Central Europe (U.S. \$26 billion). The study highlights that illicit fund investment into legitimate economies distorts resource allocation and crowds out lawful sectors, with the Caribbean identified as the primary net destination for laundered cocaine profits (approximately U.S. \$6 billion in net inflows).

[2] Benchaji, I., Douzi, S., and El Ouahidi, B., "**Predicting Credit Card Transaction Fraud Using Machine Learning Algorithms**," Journal of Advances in Information Technology, 2018. This paper evaluates five supervised machine learning models — logistic regression, neural networks, random forest, boosted tree, and support vector machines — applied to real credit card transaction data for fraud detection. The boosted tree model achieved the highest fraud detection rate (FDR = 49.83%). The study covers the full model development pipeline including data exploration, cleaning, feature engineering, and selection, and concludes that similar methodologies can be extended to fraud detection in insurance and telecommunications domains.

[3] Bengio, Y., Courville, A., and Vincent, P., "**Representation Learning: A Review and New Perspectives**," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 35, no. 8, pp. 1798–1828, Aug. 2013. This paper provides a comprehensive review of unsupervised feature learning and deep learning, covering probabilistic models, autoencoders, manifold learning, and deep networks. The authors argue that the success of machine learning is largely determined by the quality of data representations. The paper raises open research questions around objectives for learning good representations, inference methods, and geometric relationships between representation learning, density estimation, and manifold learning.

[4] Heaton, J., "**An Empirical Analysis of Feature Engineering for Predictive Modeling**," in Proc. SoutheastCon, Norfolk, VA, USA, 2016, pp. 1–6. This paper empirically investigates which types of engineered features are best suited to specific machine learning model types, including neural networks, decision trees, random forests, and gradient boosting machines. Results confirm that different models respond differently to various engineered features, and that a model's ability to synthesize certain features internally may reduce the need for manual feature engineering, though this capability varies significantly across model architectures.

[5] Coates, A., Ng, A.Y., and Lee, H., "**An Analysis of Single-Layer Networks in Unsupervised Feature Learning**," in Proc. 14th International Conference on Artificial Intelligence and Statistics (AISTATS), Fort Lauderdale, FL, USA, 2011, pp. 215–223. This paper challenges the assumption that complex algorithms and deep architectures are the primary drivers of performance in unsupervised feature learning. Applying K-means clustering, sparse autoencoders, sparse RBMs, and Gaussian mixtures on CIFAR-10, NORB, and STL datasets using only single-layer networks, the authors show that large numbers of hidden nodes and dense feature extraction are the most critical factors. K-means clustering achieved state-of-the-art results on CIFAR-10 (79.6%) and NORB (97.2%).

[6] Gold, S., "**Tracking Dirty Proceeds: Exploring Data Mining Technologies as Tools to Investigate Money Laundering**," Police Practice and Research, vol. 4, no. 2, pp. 163–178, 2003. This paper examines

how data mining technologies can enhance law enforcement's ability to detect and prevent money laundering. It reviews traditional investigative approaches that rely on informant information and manual analysis of financial records, noting their high resource consumption. The paper argues that data mining and AI-based techniques are well-suited to uncovering hidden patterns in large-scale financial datasets, offering more timely and accurate investigative leads.

[7] Senator, T.E. et al., "**Financial Crimes Enforcement Network AI System (FAIS): Identifying Potential Money Laundering from Reports of Large Cash Transactions**," AI Magazine, vol. 16, no. 4, pp. 21–39, 1995. This paper describes the FAIS system deployed at FinCEN since March 1993, processing approximately 200,000 transactions per week. FAIS integrates rule-based reasoning and a blackboard architecture, shifting analysis from a transaction-oriented to a subject-oriented perspective. The system generated over 400 investigative support reports linked to more than U.S. \$1 billion in potentially laundered funds.

[8] Jayasree, V. and Balan, R.V.S., "**Detecting Money Laundering and Terrorist Financing via Data Mining**," International Journal of Computer Applications, vol. 59, no. 3, pp. 10–14, Dec. 2012. This paper surveys data mining approaches for detecting money laundering across its three stages: placement, layering, and integration. The authors present a client profiling methodology using clustering and classification rule generation applied to real-world financial institution data. An agent-based architecture is proposed in which generated rules are embedded into intelligent agents responsible for flagging suspicious transactions.

[9] Chen, Z. et al., "**An Outlier Detection Model Based on Cross Datasets Comparison for Financial Surveillance**," in Proc. 2nd International Conference on Computer Science and Network Technology (ICCSNT), Changchun, China, 2012. This paper proposes a cross-dataset outlier detection model that compares transactions against both individual account history and peer group behavior. A distance-based model with an approximation algorithm is introduced to optimize deviation computation. Experiments on real bank data blended with synthetic outlier cases demonstrate reduced false positive rates and improved discriminative accuracy in money laundering detection.

[10] He, X., Deng, K., and Li, Y., "**Application of Cluster-Based Local Outlier Factor Algorithm in Anti-Money Laundering**," in Proc. IEEE International Conference on Intelligence and Security Informatics (ISI), Beijing, China, 2010. This paper introduces the Cluster-Based Local Outlier Factor (CBLOF) algorithm to identify suspicious money laundering transactional behavioral patterns. Combining unsupervised clustering with local outlier detection, the algorithm is validated on authentic and synthetic datasets, demonstrating effectiveness and scalability for anti-money laundering compliance in financial institutions.

[11] Coates, A., Ng, A.Y., and Lee, H., "**An Analysis of Single-Layer Networks in Unsupervised Feature Learning**," in Proc. 14th International Conference on Artificial Intelligence and Statistics (AISTATS), Fort Lauderdale, FL, USA, 2011, pp. 215–223. This paper challenges the assumption that complex algorithms and deep architectures are the primary drivers of performance in unsupervised feature learning. Applying K-means clustering, sparse autoencoders, sparse RBMs, and Gaussian mixtures on CIFAR-10, NORB, and STL datasets using only single-layer networks, the authors show that large numbers of hidden nodes and dense feature extraction are the most critical factors. K-means clustering achieved state-of-the-art results on CIFAR-10 (79.6%) and NORB (97.2%).

[12] Gold, S., "**Tracking Dirty Proceeds: Exploring Data Mining Technologies as Tools to Investigate Money Laundering**," Police Practice and Research, vol. 4, no. 2, pp. 163–178, 2003. This paper examines how data mining technologies can enhance law enforcement's ability to detect and prevent money laundering. It reviews traditional investigative approaches that rely on informant information and manual analysis of financial records, noting their high resource consumption.

3.1 Existing System

When just time-frequency characteristics are used, the F-score increases to 59.05% and the false positive rate drops to 14.9 percentage points. The false positive rate drops to 11.85% and the F-score rises to 74.06% when transaction and CRM features are taken into account.

3.2 Limitations of Existing System

1. Low prediction accuracy.
2. Security is less.
3. High false positives.
4. Manual and slow investigation process.
5. Cannot detect new laundering techniques.

3.3 Proposed System

The term "money laundering" (ML) refers to the process by which unlawful earnings are made to appear more legitimate so that they can be utilized for lawful purposes again. All significant criminal activity is linked through ML, including drug and human trafficking, terrorism, extortion, kidnap-for-ransom, bribery, embezzlement, tax fraud, corruption, and a wide variety of other offences (sometimes called predicate offences). Although it is hard to give an exact assessment of the scale of such a complicated underground industry, the International Monetary Fund (IMF) estimates that up to \$2 trillion USD is laundered annually via financial institutions throughout the world, making ML one of the world's largest marketplaces.

3.4 Advantages of Proposed System

1. High accuracy and improved security.
2. Low false positives.
3. Automatic detection using FFT + Random.
4. Faster and more reliable suspicious activity detection.

Modules Information:

To implement this project we have designed following modules

- 1) Upload Anti-Money Laundering Dataset: using this module we will upload dataset to application and then plot graph with normal and money laundering transaction count
- 2) Preprocess Dataset: dataset contains missing and non-numeric data so Random Forest will accept only numeric data so by applying Label Encoder class we are converting non-numeric data into numeric data. Label encoder assign unique ID to each non-numeric data
- 3) Run Random Forest with Transaction: processed data will be split into train and test where application will used 80% dataset to train Random Forest on transaction data and then this trained model will be applied on 20% test data to calculate FSCORE, True Positive and Negative Rate.

- 4) **Run With Transaction & Time Frequency:** using this module we will convert transaction data into Time Frequency by applying FFT algorithm and then this Time Frequency transaction data get trained with Random Forest to get FSCORE, TPR and FPR values.
- 5) **Comparison Graph:** using this module we will plot FSCORE Random Forest graph between Transaction data and Time Frequency FFT data.
- 6) **Predict Money Laundering from Test Data:** using this module we will upload test data and then apply Random Forest model to predict weather test data contains normal transaction features or Money-Laundering features.

SDLC (Umbrella Model):

Fig 1. SDLC (Umbrella Model)

Stages in SDLC:

- ◆ Analysis
- ◆ Designing
- ◆ Coding
- ◆ Testing
- ◆ Maintenance

Requirements Gathering stage:

The requirements gathering process takes as its input the goals identified in the high-level requirements section of the project plan. Each goal will be refined into a set of one or more requirements. These requirements define the major functions of the intended application, define operational data areas and reference data areas, and define the initial data entities. Major functions include critical processes to be managed, as well as mission critical inputs, outputs and reports. A user class hierarchy is developed and associated with these major functions, data areas, and data entities. Each of these definitions is termed a Requirement. Requirements are identified by unique requirement identifiers and, at minimum, contain a requirement title and textual description.

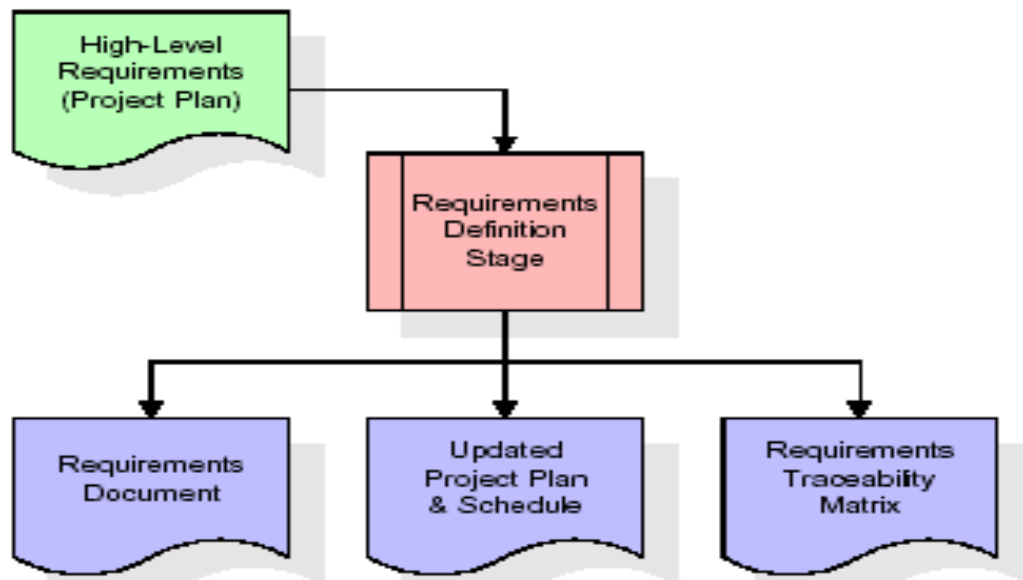


Fig 2: Requirements Gathering Stage

These requirements are fully described in the primary deliverables for this stage: the Requirements Document and the Requirements Traceability Matrix (RTM). The requirements document contains complete descriptions of each requirement, including diagrams and references to external documents as necessary. Note that detailed listings of database tables and fields are *not* included in the requirements document.

The title of each requirement is also placed into the first version of the RTM, along with the title of each goal from the project plan. The purpose of the RTM is to show that the product components developed during each stage of the software development lifecycle are formally connected to the components developed in prior

stages.

In the requirements stage, the RTM consists of a list of high-level requirements, or goals, by title, with a listing of associated requirements for each goal, listed by requirement title. In this hierarchical listing, the RTM shows that each requirement developed during this stage is formally linked to a specific product goal. In this format, each requirement can be traced to a specific product goal, hence the term requirements traceability.

The outputs of the requirements definition stage include the requirements document, the RTM, and an updated project plan.

- ◆ Feasibility study is all about identification of problems in a project.
- ◆ No. of staff required to handle a project is represented as Team Formation, in this case only modules are individual tasks will be assigned to employees who are working for that project.
- ◆ Project Specifications are all about representing of various possible inputs submitting to the server and corresponding outputs along with reports maintained by administrator.

Analysis Stage:

The planning stage establishes a bird's eye view of the intended software product, and uses this to establish the basic project structure, evaluate feasibility and risks associated with the project, and describe appropriate management and technical approaches.

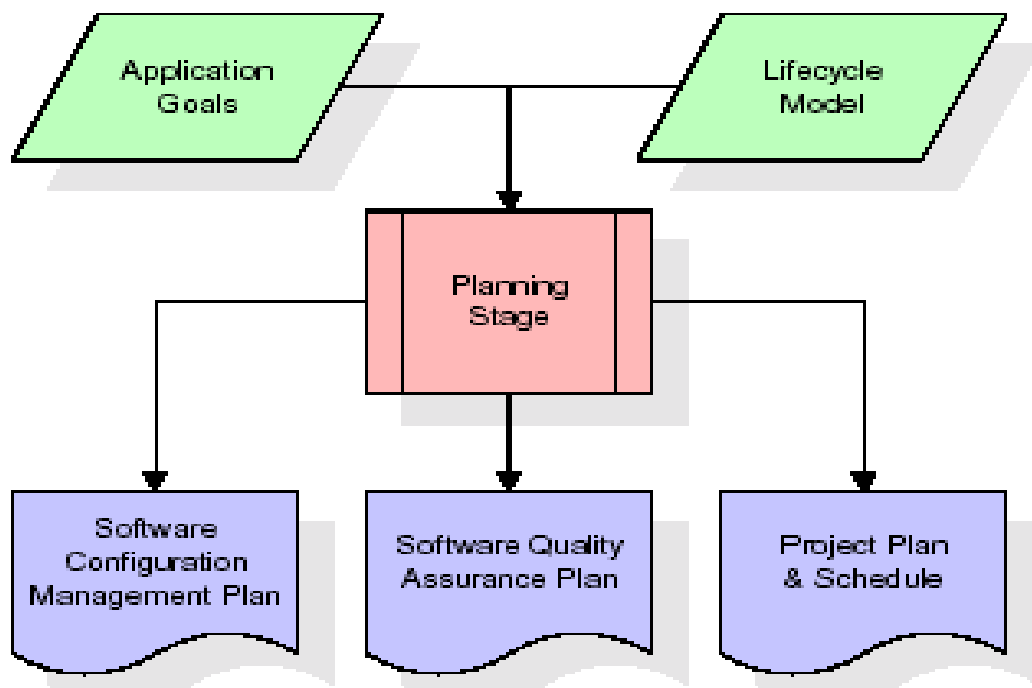


Fig 3: Analysis Stage

The most critical section of the project plan is a listing of high-level product requirements, also referred to as goals. All of the software product requirements to be developed during the requirements definition stage flow from one or more of these goals. The minimum information for each goal consists of a title and textual description, although additional information and references to external documents may be included. The outputs of the project planning stage are the configuration management plan, the quality assurance plan, and the project plan and schedule, with a detailed listing of scheduled activities for the upcoming Requirements stage, and high level estimates of effort for the out stages.

Designing Stage:

The design stage takes as its initial input the requirements identified in the approved requirements document. For each requirement, a set of one or more design elements will be produced as a result of interviews, workshops, and/or prototype efforts. Design elements describe the desired software features in detail, and generally include functional hierarchy diagrams, screen layout diagrams, tables of business rules, business process diagrams, pseudo code, and a complete entity-relationship diagram with a full data dictionary. These design elements are intended to describe the software in sufficient detail that skilled programmers may develop the software with minimal additional input.

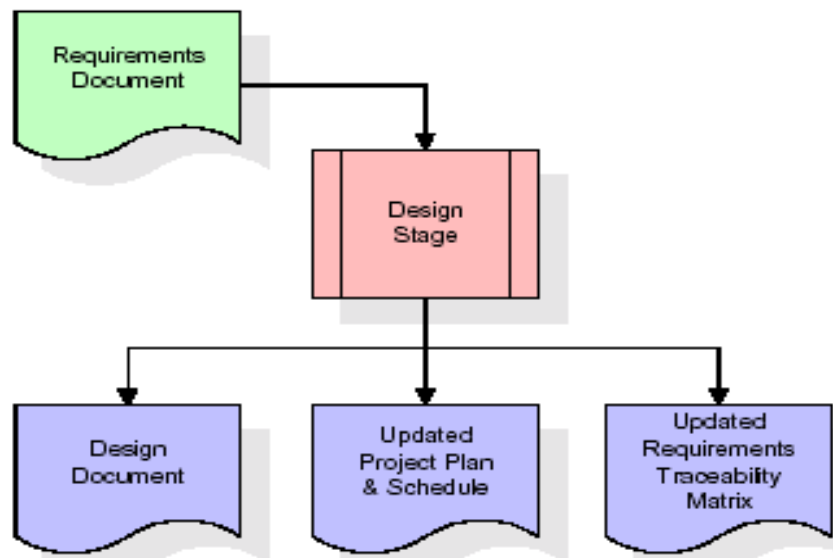


Fig 4: Designing Stage

When the design document is finalized and accepted, the RTM is updated to show that each design element is formally associated with a specific requirement. The outputs of the design stage are the design document, an updated RTM, and an updated project plan.

Development (Coding) Stage:

The development stage takes as its primary input the design elements described in the approved design document. For each design element, a set of one or more software artifacts will be produced. Software artifacts

include but are not limited to menus, dialogs, and data management forms, data reporting formats, and specialized procedures and functions. Appropriate test cases will be developed for each set of functionally related software artifacts, and an online help system will be developed to guide users in their interactions with the software.

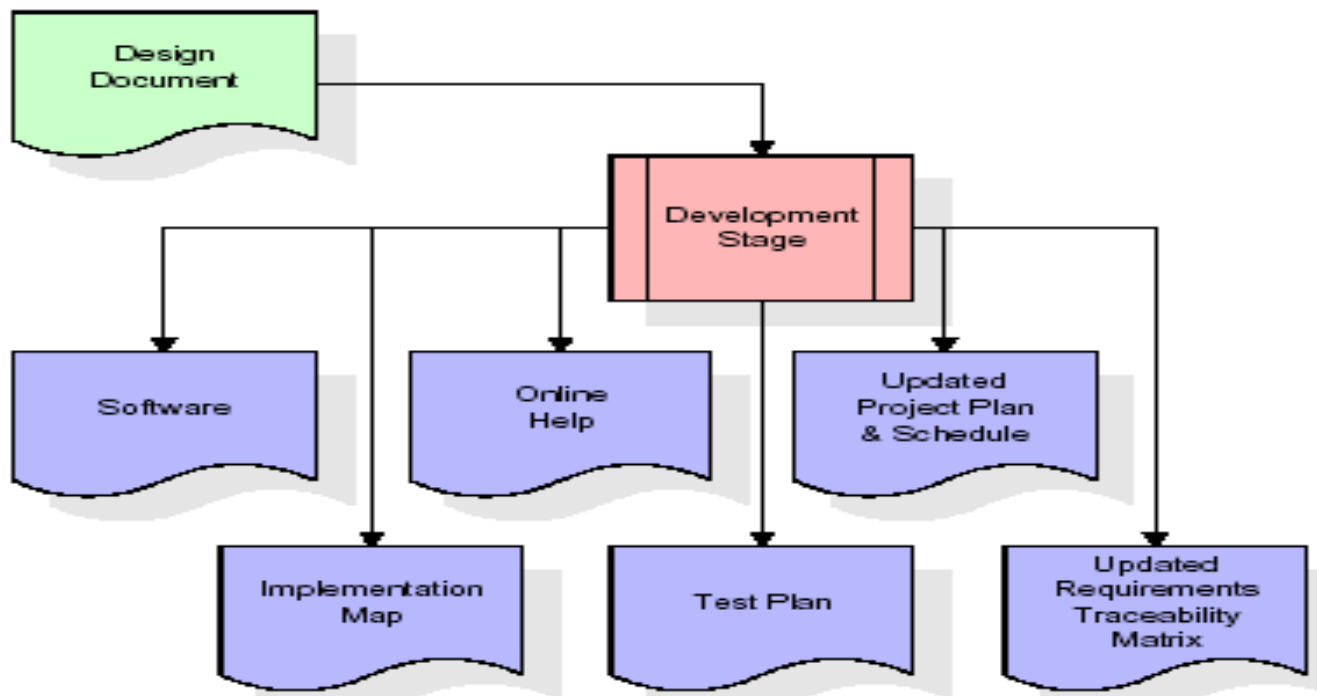


Fig 5: Development Stage

The RTM will be updated to show that each developed artifact is linked to a specific design element, and that each developed artifact has one or more corresponding test case items. At this point, the RTM is in its final configuration. The outputs of the development stage include a fully functional set of software that satisfies the requirements and design elements previously documented, an online help system that describes the operation of the software, an implementation map that identifies the primary code entry points for all major system functions, a test plan that describes the test cases to be used to validate the correctness and completeness of the software, an updated RTM, and an updated project plan.

Integration & Test Stage:

During the integration and test stage, the software artifacts, online help, and test data are migrated from the development environment to a separate test environment. At this point, all test cases are run to verify the correctness and completeness of the software. Successful execution of the test suite confirms a robust and complete migration capability.

During this stage, reference data is finalized for production use and production users are identified and linked to their appropriate roles. The final reference data (or links to reference data source files) and production user list are compiled into the Production Initiation Plan.

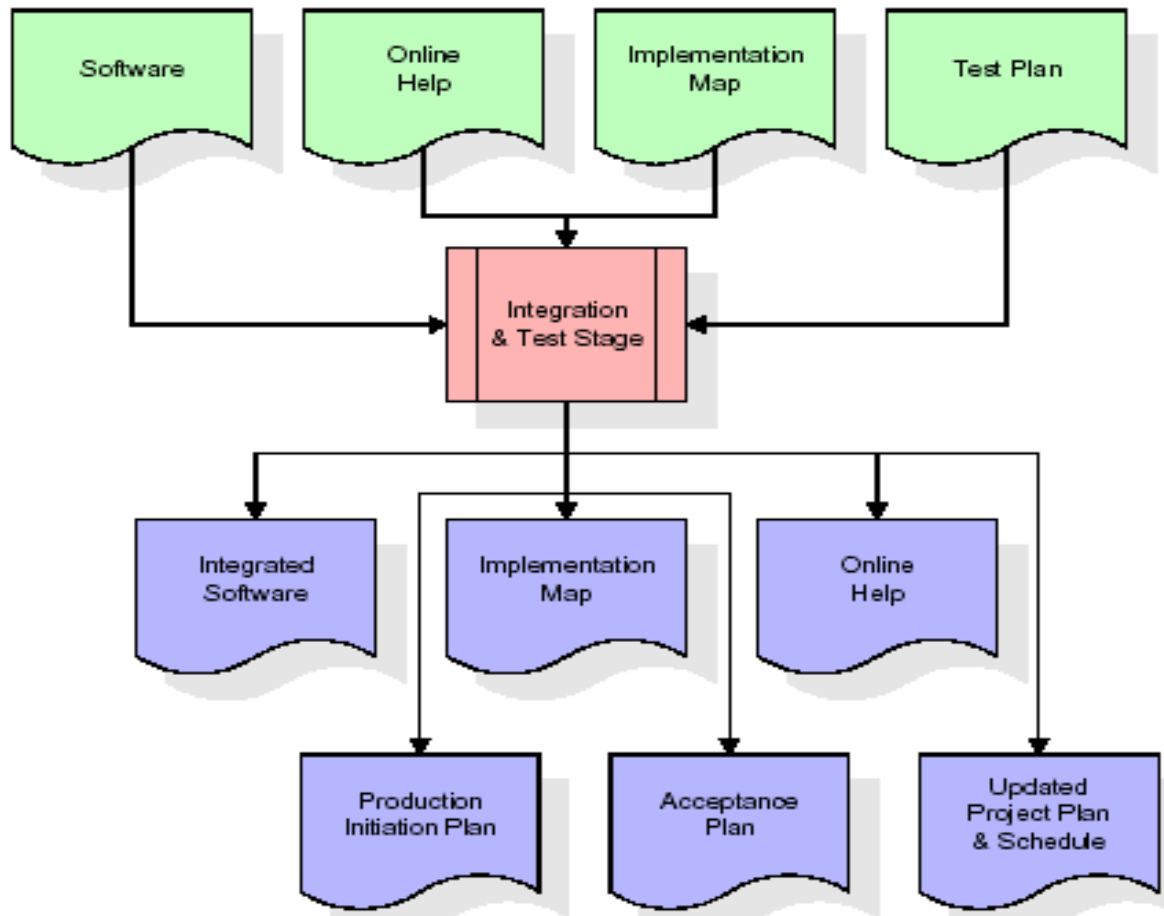


Fig 6: Integration & Test Stage

The outputs of the integration and test stage include an integrated set of software, an online help system, an implementation map, a production initiation plan that describes reference data and production users, an acceptance plan which contains the final suite of test cases, and an updated project plan.

♦ Installation & Acceptance Test:

During the installation and acceptance stage, the software artifacts, online help, and initial production data are loaded onto the production server.

At this point, all test cases are run to verify the correctness and completeness of the software. Successful execution of the test suite is a prerequisite to acceptance of the software by the customer.

After customer personnel have verified that the initial production data load is correct and the test suite has been executed with satisfactory results, the customer formally accepts the delivery of the software.

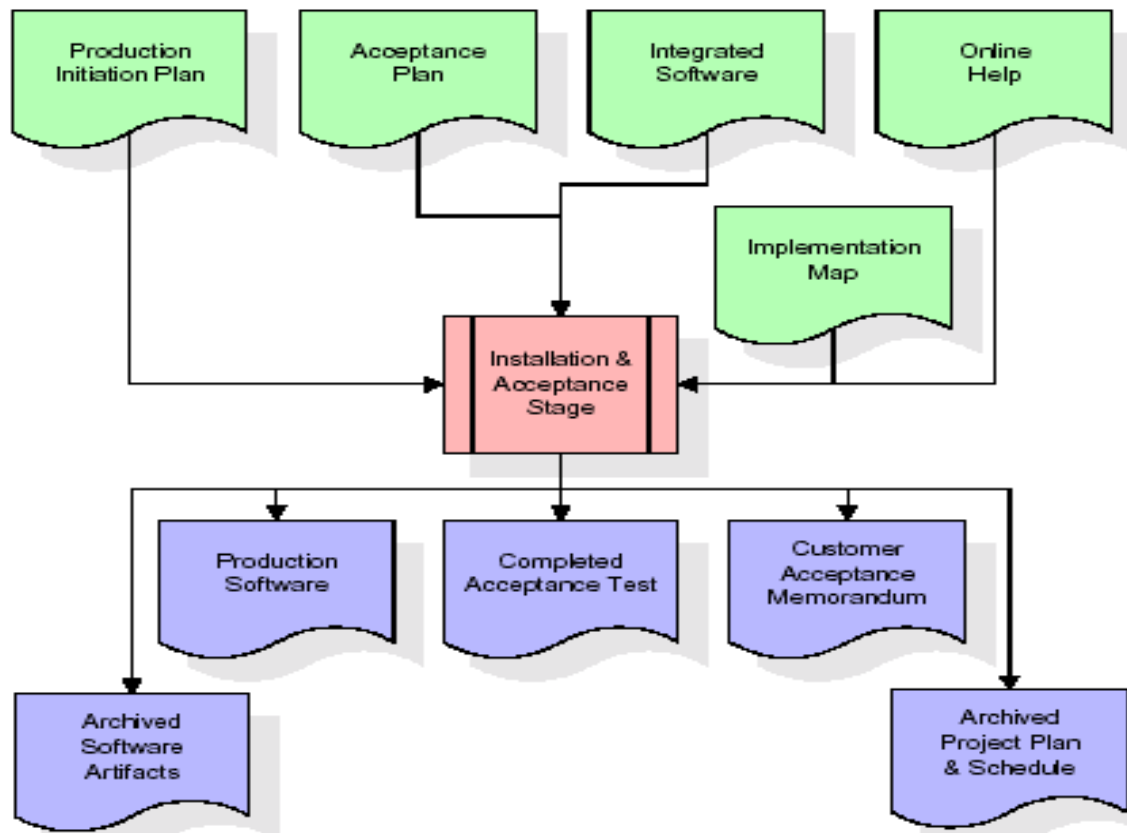


Fig 7: Installation & Acceptance Test

The primary outputs of the installation and acceptance stage include a production application, a completed acceptance test suite, and a memorandum of customer acceptance of the software.

Finally, the PDR enters the last of the actual labor data into the project schedule and locks the project as a permanent project record. At this point the PDR "locks" the project by archiving all software items, the implementation map, the source code, and the documentation for future reference.

Maintenance:

Outer rectangle represents maintenance of a project, Maintenance team will start with requirement study, understanding of documentation later employees will be assigned work and they will undergo training on that particular assigned category. For this life cycle there is no end, it will be continued so on like an umbrella (no ending point to umbrella sticks).

4. REQUIREMENT SPECIFICATION

Overall Description:

A Software Requirements Specification (SRS) – a [requirements specification](#) for a [software system](#) is a complete description of the behavior of a system to be developed. It includes a set of [use cases](#) that describe all the interactions the users will have with the software. In addition to use cases, the SRS also contains non-functional requirements. [Nonfunctional requirements](#) are requirements which impose constraints on the design or implementation (such as [performance engineering](#) requirements, [quality](#) standards, or design constraints).

System requirements specification: A structured collection of information that embodies the requirements of a system. A [business analyst](#), sometimes titled [system analyst](#), is responsible for analyzing the business needs of their clients and stakeholders to help identify business problems and propose solutions. Within the [systems development lifecycle](#) domain, the BA typically performs a liaison function between the business side of an enterprise and the information technology department or external service providers. Projects are subject to three sorts of requirements:

- [Business requirements](#) describe in business terms what must be delivered or accomplished to provide value.
- Product requirements describe properties of a system or product (which could be one of several ways to accomplish a set of business requirements.)
- Process requirements describe activities performed by the developing organization. For instance, process requirements could specify .Preliminary investigation examine project feasibility, the likelihood the system will be useful to the organization. The main objective of the feasibility study is to test the Technical, Operational and Economical feasibility for adding new modules and debugging old running system. All system is feasible if they are unlimited resources and infinite time. There are aspects in the feasibility study portion of the preliminary investigation:

ECONOMIC FEASIBILITY

A system can be developed technically and that will be used if installed must still be a good investment for the organization. In the economical feasibility, the development cost in creating the system is evaluated against the ultimate benefit derived from the new systems. Financial benefits must equal or exceed the costs. The system is economically feasible. It does not require any addition hardware or software.

Since the interface for this system is developed using the existing resources and technologies available at NIC, There is nominal expenditure and economical feasibility for certain.

OPERATIONAL FEASIBILITY

Proposed projects are beneficial only if they can be turned out into information system. That will meet the organization's operating requirements. Operational feasibility aspects of the project are to be taken as an

important part of the project implementation. This system is targeted to be in accordance with the above-mentioned issues. Beforehand, the management issues and user requirements have been taken into consideration.

So there is no question of resistance from the users that can undermine the possible application benefits. The well-planned design would ensure the optimal utilization of the computer resources and would help in the improvement of performance status.

TECHNICAL FEASIBILITY

Earlier no system existed to cater to the needs of ‘Secure Infrastructure Implementation System’. The current system developed is technically feasible. It is a web based user interface for audit workflow at NIC-CSD. Thus it provides an easy access to .the users.

The database’s purpose is to create, establish and maintain a workflow among various entities in order to facilitate all concerned users in their various capacities or roles.

Permission to the users would be granted based on the roles specified. Therefore, it provides the technical guarantee of accuracy, reliability and security.

► External Interface Requirements

◆ User Interface

The user interface of this system is a user friendly python Graphical User Interface.

◆ Hardware Interfaces

The interaction between the user and the console is achieved through python capabilities.

◆ Software Interfaces

The required software is python.

◆ SYSTEM REQUIREMENT:

HARDWARE REQUIREMENTS:

- Processor - Intel i3(min)
- Speed - 1.1 Ghz
- RAM - 4GB(min)
- Hard Disk - 500 GB
- Key Board - Standard Windows Keyboard
- Mouse - Two or Three Button Mouse
- Monitor - SVGA

SOFTWARE REQUIREMENTS:

- Operating System - Windows10(min)
- Programming Language - Python (3.7.0)

5. PROPOSED METHODOLOGY

The proposed methodology aims to enhance suspicious transaction detection in Anti-Money Laundering (AML) systems by introducing a time-frequency–based feature engineering approach. Traditional AML solutions rely heavily on manually engineered transactional and CRM features, which are often insufficient for detecting complex laundering behaviors. The methodology presented in this project reduces the dependency on manual feature creation and improves prediction performance by transforming transactional data into meaningful representations in both time and frequency domains.

This methodology is structured into three major stages:

1. Time-Frequency Feature Engineering,
2. Machine Learning Model Development, and
3. Evaluation on Real Banking Data.

Time–Frequency Feature Engineering

Feature engineering plays a crucial role in any machine learning–based AML system. However, financial datasets typically contain thousands of potential features, making the process highly time- consuming and prone to error. To address this limitation, the proposed method applies Time– Frequency (TF) Analysis as the core feature extraction strategy

In this stage, raw transaction values are transformed using the Fast Fourier Transform (FFT), which provides a time-frequency representation of each transaction sequence.

From this representation, statistically rich features are extracted—such as mean, variance, skewness, kurtosis, and other 2-D TF characteristics. These features capture hidden behavioral patterns that are often undetectable in the time domain alone.

By relying on TF-based features, the methodology significantly reduces manual effort in feature selection while producing a compact and discriminative feature set. These features have demonstrated strong capability in distinguishing suspicious entities from normal transactional behavior.

Machine Learning Model Development

Once the TF feature set is prepared, the next stage involves building a machine learning model capable of identifying suspicious transactions with high accuracy. In this work, the Random Forest classifier is selected due to its robustness, interpretability, and proven performance in fraud-related applications.

The model is trained using two different configurations:

1. Transaction-only features, and
2. Transaction + Time-Frequency features.

To further enhance model performance, Simulated Annealing is employed for hyperparameter tuning. This optimization technique ensures that the Random Forest achieves the best possible balance between precision and false-positive reduction.

Model evaluation metrics include F-Score, Area Under the Curve (AUC), True Positive Rate (TPR), and False Positive Rate (FPR). The inclusion of TF features leads to substantial performance improvements, especially in reducing false positives—one of the most critical challenges in AML systems.

Evaluation Using Real Banking Data

The final stage of the proposed methodology involves validating the model on real-world banking datasets. This step ensures that the approach is practical and effective in operational AML environments.

During evaluation, both models—transaction-only and TF-enhanced—are tested on authentic financial transaction data. The results clearly demonstrate that TF features provide superior discriminatory power. In particular, the TF-enhanced Random Forest model:

- Achieves higher F-Scores,
- Shows improved AUC values,
- Reduces false positives significantly, and
- Detects suspicious patterns that traditional models fail to capture.

This real-data evaluation confirms the effectiveness of time-frequency analysis in AML and highlights its potential for large-scale deployment in financial institutions.

Summary of the Methodology

The proposed methodology provides a novel, practical and data-driven solution to suspicious transaction detection. By integrating time-frequency analysis with machine learning, the approach enhances model accuracy, reduces feature engineering workload, and improves operational efficiency. The three-stage methodology can be summarized as follows:

1. Time-Frequency Feature Engineering

- Converts raw transactions into TF domain via FFT.
- Extracts statistical TF features for improved discrimination.

2. Model Development and Optimization

- Trains a Random Forest classifier using TF-enhanced features.
- Uses simulated annealing for hyperparameter tuning.

3. Real-World Evaluation

- Validates the methodology on actual banking datasets.
- Demonstrates significant improvements in precision and false-positive reduction.

6. SYSTEM DESIGN

6.1. ARCHITECTURE

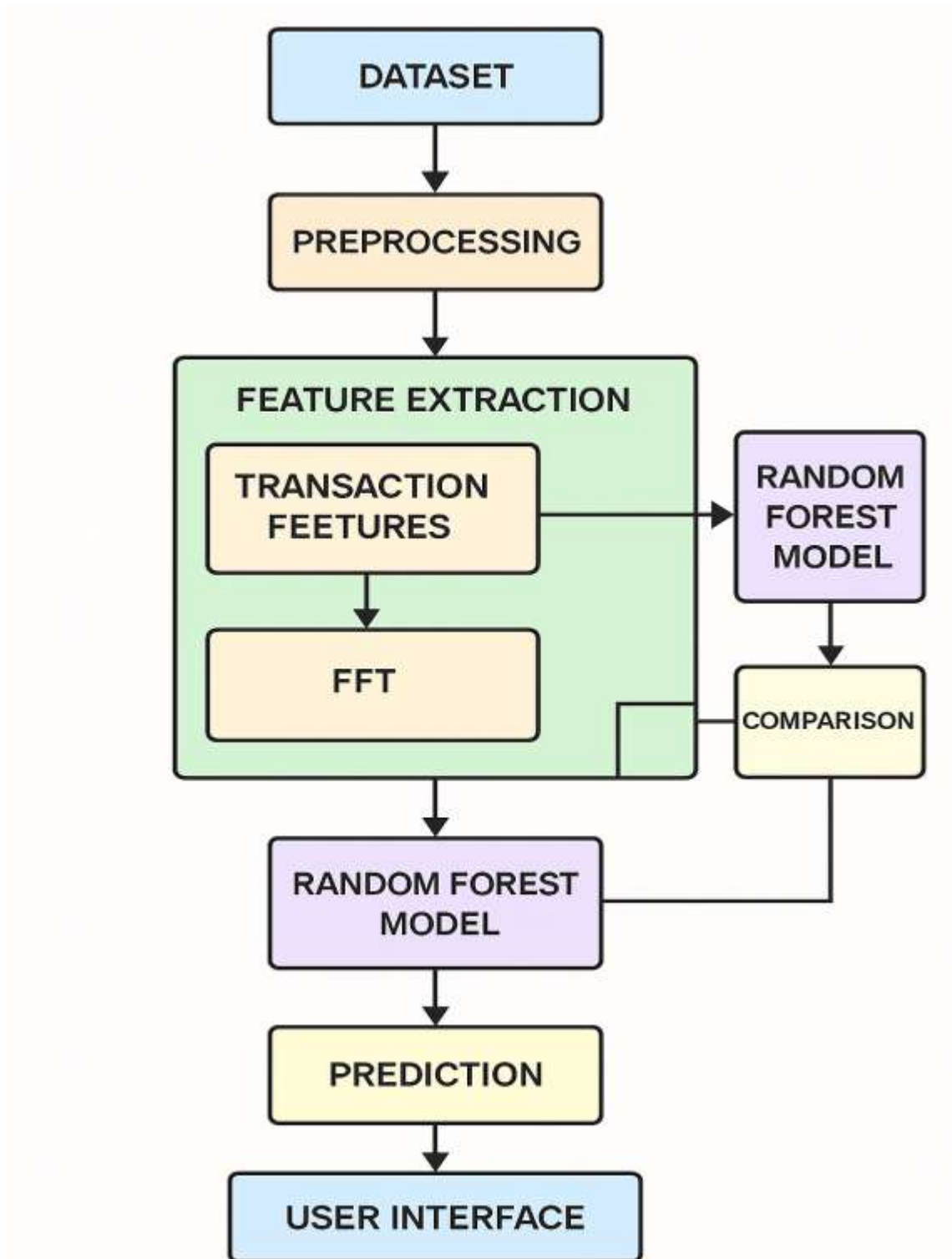


Fig 8: Architecture

The architecture of the proposed Anti-Money Laundering (AML) detection system follows a modular, high-level pipeline designed to process transaction data, extract meaningful features, train machine learning models, and generate accurate predictions. The architecture is derived from the modules and workflow described in the project documentation.

1. Dataset Input Layer

The system begins with the dataset upload module, where financial transaction data is provided as input. This raw dataset contains various transaction attributes needed for analysis.

2. Preprocessing Layer

Before applying machine learning algorithms, the dataset undergoes preprocessing. This includes:

- Label Encoding to convert non-numeric values into numeric form.
- Cleaning missing values.
- Normalizing and preparing the dataset for model training.

3. Feature Extraction Layer

The system supports two types of feature extraction:

- a. Transaction Features – Basic numerical transaction attributes used directly for Random Forest training.
- b. Time-Frequency (FFT) Features – The system converts transaction records into time-frequency representations using Fast Fourier Transform (FFT). These extracted statistical features (mean, variance, skewness, kurtosis, etc.) significantly improve predictive accuracy.

4. Machine Learning Layer

Two Random Forest models are trained:

- Random Forest using transaction features.
- Random Forest using FFT-generated time-frequency features.

The model trained with FFT features achieves higher FSCORE, TPR, and reduced false positives.

5. Comparison Module

This module compares:

- FSCORE of Transaction Data Model
- FSCORE of FFT-Based Model

The comparison validates improved performance with time-frequency features.

6. Prediction Layer

A user-uploaded test dataset passes through preprocessing and is evaluated by the best-performing model. The system predicts whether a transaction is:

- Normal
- Suspicious (Money-Laundering)

7. User Interface Layer

A Python GUI allows users to upload data, preprocess, train models, compare performance, and perform predictions in a simple and interactive manner.

Overall, the architecture ensures high accuracy, reduced false positives, automated feature extraction, and improved security in detecting suspicious financial activities.

6.2 UML DIAGRAM

UML Representation for Time-Frequency Based Suspicious Activity Detection System

• User Model View

- I. This view represents the AML detection system from the user's perspective.
- II. The user (AML analyst, investigator, or system operator) interacts with the system to upload Transaction datasets, preprocess the data, train machine learning models, generate comparison graphs, and predict suspicious transactions.
- III. The use case scenarios include: uploading data, performing preprocessing, executing Random Forest classification, applying time-frequency feature extraction (FFT), visualizing results, and obtaining predictions.

• Structural Model View

- I. This model identifies the internal data structures and processing components involved in the

suspicious activity detection system.

II. It focuses on the static representation of system elements such as:

- Dataset Manager
- Preprocessing module (Label Encoding, Missing Value Handling)
- Time-Frequency Transformer (FFT feature extractor)
- Random Forest Training Engine
- Prediction Service
- Graph/Visualization Manager

III. It describes how these components are organized and what data each component stores and processes.

Behavioral Model View

i. This view represents the dynamic behavior of the system by showing how data flows and how components interact-during-execution.

ii. It illustrates the sequence of operations such as:

- Loading the dataset
- Preprocessing and feature engineering
- Training the model using transaction-only and time-frequency features
- Comparing F-scores and performance metrics
- Predicting suspicious activity from test data

iii. It shows the collaboration among structural components (Dataset Manager, Preprocessor, RF Trainer, FFT module, Prediction Engine) and their interactions during system operation.

Implementation Model View

i. This view represents how the identified structural and behavioral elements are realized in the actual implementation.

ii. The system is developed using Python and makes use of several libraries such as:

- pandas, numpy for data handling
- scikit-learn for Random Forest modeling
- scipy (FFT) for time-frequency feature extraction
- tkinter for the GUI
- matplotlib for plotting performance graphs

iii. The implementation model maps conceptual components to Python modules, functions, and classes used to build the working solution.

• Environmental Model View

- i. This view represents the environment in which the AML detection system operates.
- ii. The system runs on a standard computing device (Windows OS), with Python installed and required libraries-configured.
- iii. It interacts with the external environment through:

- Uploaded CSV files containing transaction data
 - Test datasets for prediction
 - Hardware resources (CPU, RAM) to perform FFT and machine learning computations
- iv. This view also represents integration potential with various banking channels (ATM, branch, online transactions) and FIU data sources in future extensions.

1.CLASS DIAGRAM:

The class diagram is the main building block of object oriented modeling. It is used both for general conceptual modeling of the systematic of the application, and for detailed modeling translating the models into programming code. Class diagrams can also be used for data modeling.

The classes in a class diagram represent both the main objects, interactions in the application and the classes to be programmed. In the diagram, classes are represented with boxes which contain three parts:

- The upper part holds the name of the class
- The middle part contains the attributes of the class
- The bottom part gives the methods or operations the class can take or undertake



Fig 9: Class Diagram

2. USECASE DIAGRAM:

A **use case diagram** at its simplest is a representation of a user's interaction with the system and depicting the specifications of a use case. A use case diagram can portray the different types of users of a system and the various ways that they interact with the system. This type of diagram is typically used in conjunction with the textual use case and will often be accompanied by other types of diagrams as well.

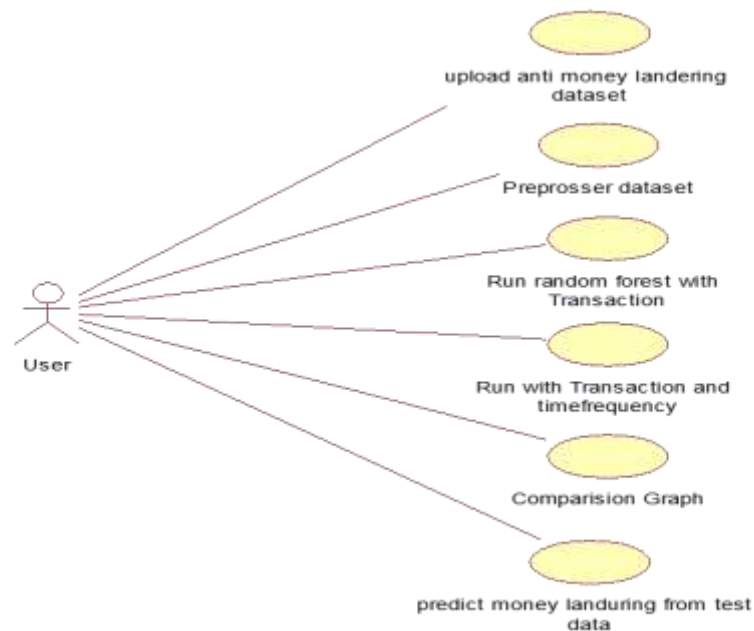


Fig 10: Usecase Diagram

3. SEQUENCE DIAGRAM:

A **sequence diagram** is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use case realizations in the Logical View of the system under development. Sequence diagrams are sometimes called **event diagrams**, **event scenarios**, and timing diagrams.

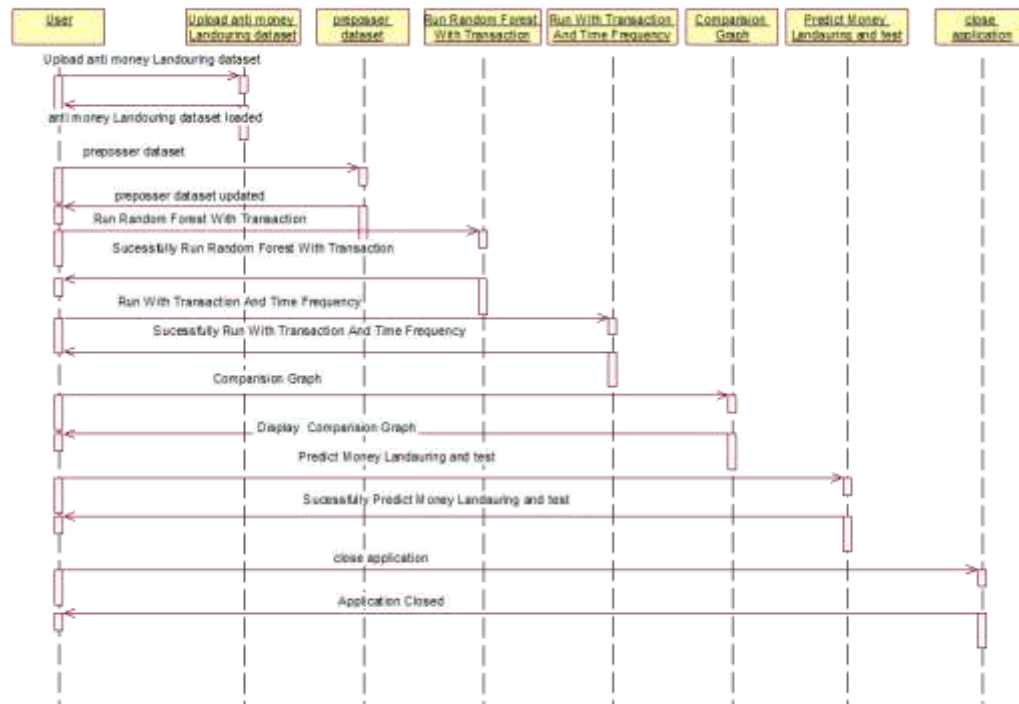


Fig 11: Sequence Diagram

4. COLLABORATION DIAGRAM:

A collaboration diagram describes interactions among objects in terms of sequenced messages. Collaboration diagrams represent a combination of information taken from class, sequence, and use case diagrams describing both the static structure and dynamic behavior of a system.

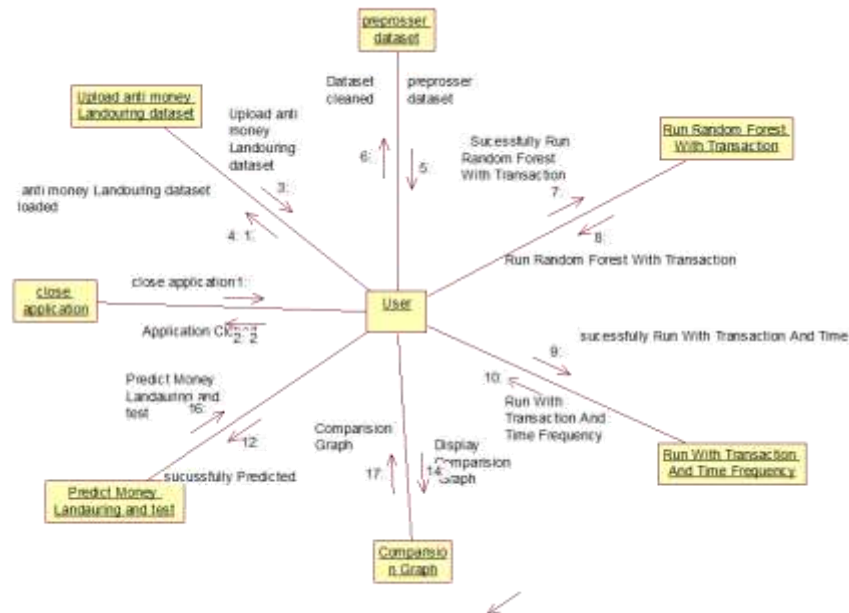


Fig 12: Collaboration Diagram

5. COMPONENT DIAGRAM:

In the Unified Modelling Language, a component diagram depicts how components are wired together to form larger components and or software systems. They are used to illustrate the structure of arbitrarily complex systems.

Components are wired together by using an assembly connector to connect the required interface of one component with the provided interface of another component. This illustrates the service consumer - service provider relationship between the two components.

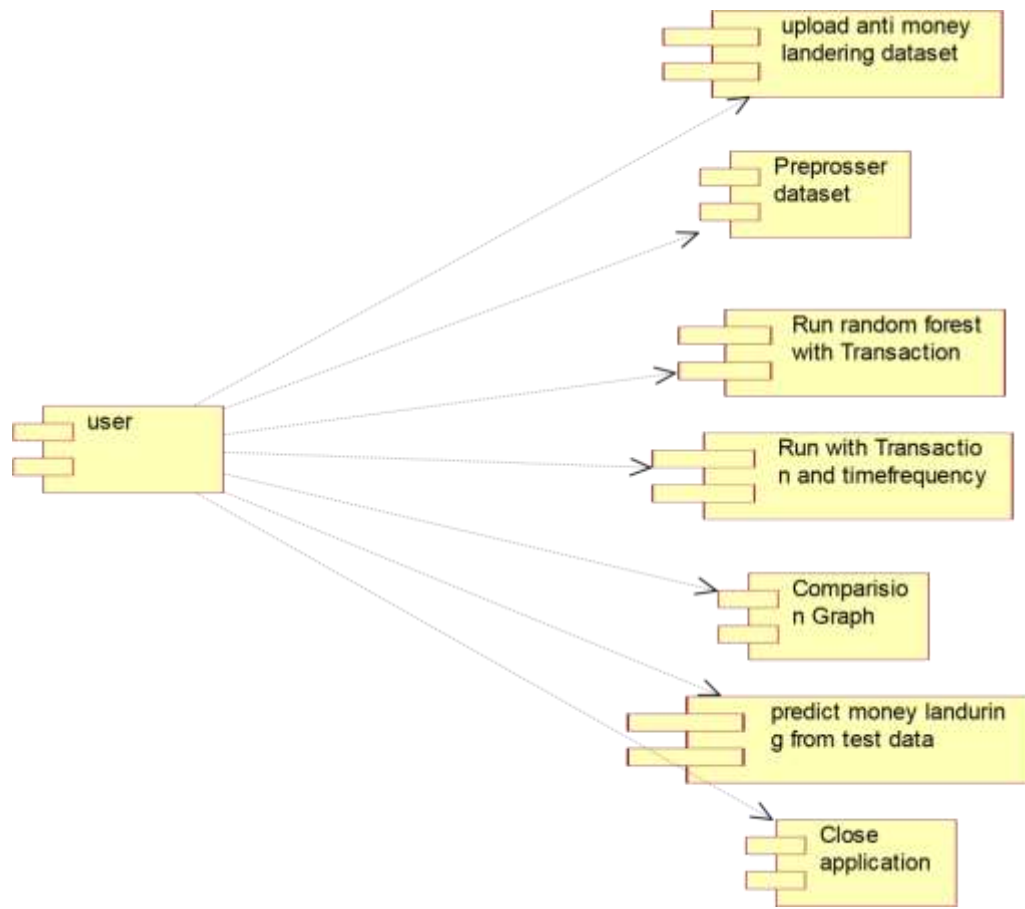


Fig 13: Component Diagram

6. DEPLOYMENT DIAGRAM:

A **deployment diagram** in the Unified Modeling Language models the *physical* deployment of artifacts on nodes. To describe a web site, for example, a deployment diagram would show what hardware components ("nodes") exist (e.g., a web server, an application server, and a database server), what software components ("artifacts") run on each node (e.g., web application, database), and how the different pieces are connected (e.g. JDBC, REST, RMI).

The nodes appear as boxes, and the artifacts allocated to each node appear as rectangles within the boxes. Nodes may have sub nodes, which appear as nested boxes. A single node in a deployment diagram may conceptually represent multiple physical nodes, such as a cluster of database servers.

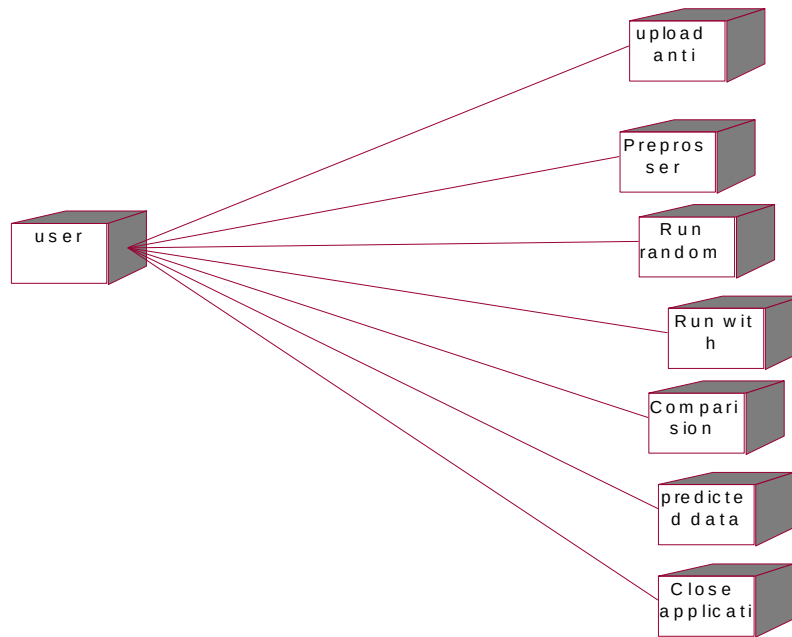


Fig 14: Deployment Diagram

ACTIVITY DIAGRAM:

Activity diagram is another important diagram in UML to describe dynamic aspects of the system. It is basically a flow chart to represent the flow from one activity to another activity. The activity can be described as an operation of the system. So the control flow is drawn from one operation to another. This flow can be sequential, branched or concurrent.

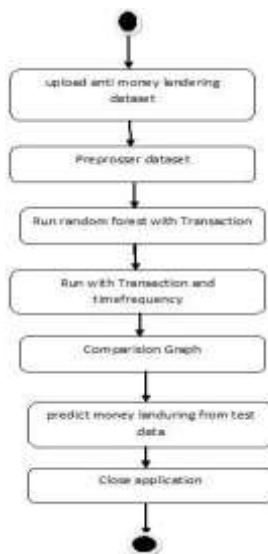


Fig 15: Activity Diagram

8. DATA FLOW DIAGRAM

Data flow diagrams illustrate how data is processed by a system in terms of inputs and outputs. Data flow diagrams can be used to provide a clear representation of any business function. The technique starts with an overall picture of the business and continues by analyzing each of the functional areas of interest. This analysis can be carried out in precisely the level of detail required. The technique exploits a method called top-down expansion to conduct the analysis in a targeted way.

As the name suggests, Data Flow Diagram (DFD) is an illustration that explicates the passage of information in a process. A DFD can be easily drawn using simple symbols. Additionally, complicated processes can be easily automated by creating DFDs using easy-to-use, free downloadable diagramming tools. A DFD is a model for constructing and analyzing information processes. DFD illustrates the flow of information in a process depending upon the inputs and outputs. A DFD can also be referred to as a Process Model. A DFD demonstrates business or technical process with the support of the outside data saved, plus the data flowing from the process to another and the end results.

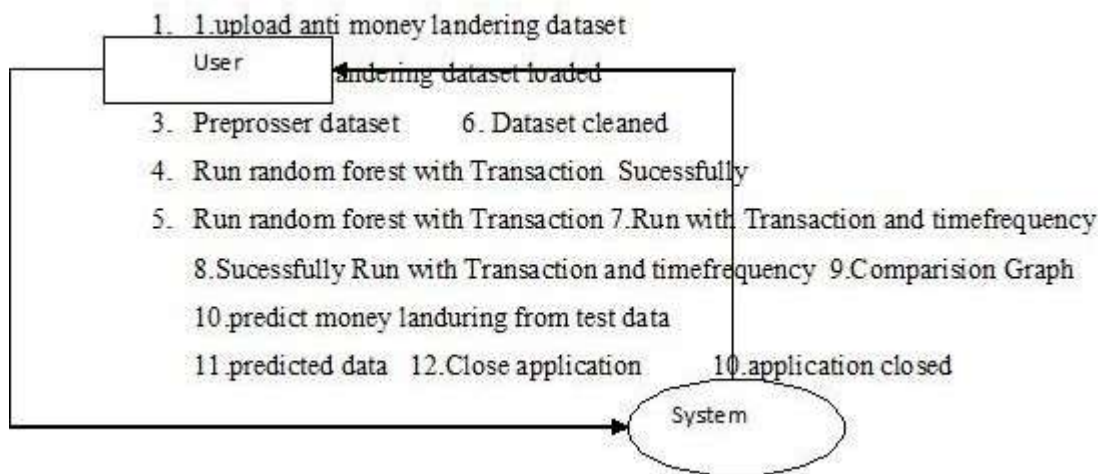


Fig 16: Data Flow

7. IMPEMETATION

7.1 Python

Python is a general-purpose language. It has wide range of applications from Web development (like: Django and Bottle), scientific and mathematical computing (Orange, SymPy, NumPy) to desktop graphical user Interfaces (Pygame, Panda3D). The syntax of the language is clean and length of the code is relatively short. It's fun to work in Python because it allows you to think about the problem rather than focusing on the syntax.

History of Python:

Python is a fairly old language created by Guido Van Rossum. The design began in the late 1980s and was first released in February 1991.

Why Python was created?

In late 1980s, Guido Van Rossum was working on the Amoeba distributed operating system group. He wanted to use an interpreted language like ABC (ABC has simple easy-to-understand syntax) that could access the Amoeba system calls. So, he decided to create a language that was extensible. This led to design of a new language which was later named Python.

Why the name Python?

No. It wasn't named after a dangerous snake. Rossum was fan of a comedy series from late seventies. The name "Python" was adopted from the same series "Monty Python's Flying Circus".

Features of Python:

1. A simple language which is easier to learn

Python has a very simple and elegant syntax. It's much easier to read and write Python programs compared to other languages like: C++, Java, C#. Python makes programming fun and allows you to focus on the solution rather than syntax. If you are a newbie, it's a great choice to start your journey with Python.

2. Free and open-source

You can freely use and distribute Python, even for commercial use. Not only can you use and distribute software's written in it, you can even make changes to the Python's source code. Python has a large community constantly improving it in each iteration.

3. Portability

You can move Python programs from one platform to another, and run it without any changes. It runs seamlessly on almost all platforms including Windows, Mac OS X and Linux.

4. Extensible and Embeddable

Suppose an application requires high performance. You can easily combine pieces of C/C++ or other languages with Python code. This will give your application high performance as well as scripting capabilities which other languages may not provide out of the box.

5. A high-level, interpreted language

Unlike C/C++, you don't have to worry about daunting tasks like memory management, garbage collection and so on. Likewise, when you run Python code, it automatically converts your code to the language your computer understands. You don't need to worry about any lower-level operations.

6. Large standard libraries to solve common tasks

Python has a number of standard libraries which makes life of a programmer much easier since you don't have to write all the code yourself. For example: Need to connect MySQL database on a Web server? You can use MySQLdb library using `import MySQLdb`. Standard libraries in Python are well tested and used by hundreds of people. So you can be sure that it won't break your application.

7. Object-oriented

Everything in Python is an object. Object oriented programming (OOP) helps you solve a complex problem intuitively. With OOP, you are able to divide these complex problems into smaller sets by creating objects.

Applications of Python:

1. Simple Elegant Syntax

Programming in Python is fun. It's easier to understand and write Python code. Why? The syntax feels natural.

Take this source code for an example:

```
a = 2
b = 3
sum = a + b
print(sum)
```

2. Not overly strict

You don't need to define the type of a variable in Python. Also, it's not necessary to add semicolon at the end of the statement. Python enforces you to follow good practices (like proper indentation). These small things can make learning much easier for beginners.

3. Expressiveness of the language

Python allows you to write programs having greater functionality with fewer lines of code. Here's a link to

the source code of Tic-tac-toe game with a graphical interface and a smart computer opponent in less than 500 lines of code. This is just an example. You will be amazed how much you can do with Python once you learn the basics.

4. Great Community and Support

Python has a large supporting community. There are numerous active forums online which can be handy if you are stuck.

7.2 Code Snippet:

DepressionDetection.py

```
from tkinter import *
import tkinter
from tkinter import filedialog
import numpy as np
from tkinter.filedialog import askopenfilename
import pandas as pd
from tkinter import simpledialog
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
from scipy.fft import fft
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
import seaborn as sns
from sklearn.metrics import confusion_matrix

main = tkinter.Tk()
main.title("A Time-Frequency Based Suspicious Activity Detection for Anti-Money Laundering")
main screen
main.geometry("1000x650")

global filename
global fscore
global X_train, X_test, y_train, y_test
global X, Y
global le1, le2, le3, dataset, rf

def upload():
```

```

global filename, dataset
filename = filedialog.askopenfilename(initialdir = "Dataset")
text.delete('1.0', END)
text.insert(END,filename+' Loaded')
dataset = pd.read_csv(filename)
text.insert(END,str(dataset.head()))
label = dataset.groupby('Label').size()
label.plot(kind="bar")
plt.title("Transaction Graph 0 Means Normal Transaction & 1 Means Money Laundering Transaction")
plt.show()

```

```

def preprocessDataset():
    global X, Y
    global le1, le2, le3, dataset
    text.delete('1.0', END)
    dataset.fillna(0, inplace = True)
    cols = ['type','nameOrig','nameDest']
    le1 = LabelEncoder()
    le2 = LabelEncoder()
    le3 = LabelEncoder()
    dataset[cols[0]] = pd.Series(le1.fit_transform(dataset[cols[0]].astype(str)))
    dataset[cols[1]] = pd.Series(le2.fit_transform(dataset[cols[1]].astype(str)))
    dataset[cols[2]] = pd.Series(le3.fit_transform(dataset[cols[2]].astype(str)))

    Y = dataset['Label'].ravel()
    dataset.drop(['Label'], axis = 1,inplace=True)
    X = dataset.values
    indices = np.arange(X.shape[0])
    np.random.shuffle(indices)
    X = X[indices]
    Y = Y[indices]
    text.insert(END,"Processed Dataset values\n\n")
    text.insert(END,str(X)+"\n\n")
    text.insert(END,"Total records found in dataset : "+str(X.shape[0])+"\n")
    text.insert(END,"Total features found in dataset : "+str(X.shape[1] + 1)+"\n")

def runTransactionRandomForest():
    global X, Y, fscore, rf
    fscore = []
    text.delete('1.0', END)

```

```

X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2)
cls = RandomForestClassifier(ccp_alpha=0.5)
cls.fit(X_train, y_train)
predict = cls.predict(X_test)
f = f1_score(y_test, predict, average='macro') * 100
fscore.append(f)
CM = confusion_matrix(y_test, predict)
TN = CM[0][0] / len(y_test)
FN = CM[1][0] / len(y_test)
TP = CM[1][1] / len(y_test)
FP = CM[0][1] / len(y_test)
cls = RandomForestClassifier()
cls.fit(X_train, y_train)
rf = cls
text.insert(END, "Random Forest FSCORE on Transaction Data : "+str(f)+"\n")
text.insert(END, "False Positive Rate (FPR) : "+str(FP)+"%\n")
text.insert(END, "False Negative Rate (FNR) : "+str(FN)+"%\n")
text.insert(END, "True Negative Rate (PPV) : "+str(TN)+"%\n")
text.insert(END, "True Positive Rate (TPR) : "+str(TP)+"%\n\n")

```

```

def runTimeFrequencyRandomForest():

```

```

    global X, Y, fscore, rf
    fft_data = fft(X)
    X = []
    for i in range(len(fft_data)):
        temp = []
        for j in range(len(fft_data[i])):
            temp.append(float(fft_data[i,j]))
        X.append(temp)
    X = np.asarray(X)

```

```

X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2)
cls = RandomForestClassifier()
cls.fit(X_train, y_train)
predict = cls.predict(X_test)
f = f1_score(y_test, predict, average='macro') * 100
fscore.append(f)
CM = confusion_matrix(y_test, predict)
TN = CM[0][0] / len(y_test)
FN = CM[1][0] / len(y_test)
TP = CM[1][1] / len(y_test)
FP = CM[0][1] / len(y_test)

```



```

text.insert(END,"Random Forest FSCORE on Transaction & Time Frequency Data : "+str(f)+"\n")
text.insert(END,"False Positive Rate (FPR) : "+str(FP)+"%\n")
text.insert(END,"False Negative Rate (FNR) : "+str(FN)+"%\n")
text.insert(END,"True Negative Rate (PPV) : "+str(TN)+"%\n")
text.insert(END,"True Positive Rate (TPR) : "+str(TP)+"%\n\n")

def predict():
    global rf, le1, le2, le3
    filename = filedialog.askopenfilename(initialdir = "Dataset")
    text.delete('1.0', END)
    dataset = pd.read_csv(filename)
    dataset.fillna(0, inplace = True)
    cols = ['type','nameOrig','nameDest']
    dataset[cols[0]] = pd.Series(le1.transform(dataset[cols[0]].astype(str)))
    dataset[cols[1]] = pd.Series(le2.transform(dataset[cols[1]].astype(str)))
    dataset[cols[2]] = pd.Series(le3.transform(dataset[cols[2]].astype(str)))
    dataset = dataset.values
    predict = rf.predict(dataset)
    print(predict)
    for i in range(len(predict)):
        if predict[i] == 0:
            text.insert(END,"Transaction data : "+str(dataset[i])+" ==> PREDICTED AS NORMAL\n\n")
        if predict[i] == 1:
            text.insert(END,"Transaction data : "+str(dataset[i])+" ==> PREDICTED AS MONEY-
LAUNDERING\n\n")

def graph():
    height = fscore
    bars = ('Transaction Data FScore','Time Frequency Data FScore')
    y_pos = np.arange(len(bars))
    plt.bar(y_pos, height)
    plt.title("Random Forest FScore on Transaction & Time Frequency Data")
    plt.xticks(y_pos, bars)
    plt.show()

def close():
    main.destroy()

font = ('times', 16, 'bold')
title = Label(main, text='A Time-Frequency Based Suspicious Activity Detection for Anti-Money
Laundering', justify=LEFT)
title.config(bg='lavender blush', fg='DarkOrchid1')

```

```

title.config(font=font)
title.config(height=3, width=120)
title.place(x=100,y=5)
title.pack()

font1 = ('times', 13, 'bold')
uploadButton = Button(main, text="Upload Anti-Money Laundering Dataset", command=upload)
uploadButton.place(x=10,y=100)
uploadButton.config(font=font1)

processButton = Button(main, text="Preprocess Dataset", command=preprocessDataset)
processButton.place(x=10,y=150)
processButton.config(font=font1)

transactionButton = Button(main, text="Run Random Forest with Transaction",
command=runTransactionRandomForest)
transactionButton.place(x=10,y=200)
transactionButton.config(font=font1)

tfButton = Button(main, text="Run With Transaction & Time Frequency",
command=runTimeFrequencyRandomForest)
tfButton.place(x=10,y=250)
tfButton.config(font=font1)
graphButton = Button(main, text="Comparison Graph", command=graph)
graphButton.place(x=10,y=300)
graphButton.config(font=font1)

predictButton = Button(main, text="Predict Money Laundering from Test Data", command=predict)
predictButton.place(x=10,y=350)
predictButton.config(font=font1)
closeButton = Button(main, text="Close Application", command=close)
closeButton.place(x=10,y=400)
closeButton.config(font=font1)

font1 = ('times', 12, 'bold')
text=Text(main,height=20,width=160)
scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set)
text.place(x=400,y=100)
text.config(font=font1)
main.config(bg='light coral')
main.mainloop()

```

8. TESTING

Implementation and Testing:

Implementation is one of the most important tasks in project is the phase in which one has to be cautions because all the efforts undertaken during the project will be very interactive. Implementation is the most crucial stage in achieving successful system and giving the users confidence that the new system is workable and effective. Each program is tested individually at the time of development using the sample data and has verified that these programs link together in the way specified in the program specification. The computer system and its environment are tested to the satisfaction of the user.

Implementation

The implementation phase is less creative than system design. It is primarily concerned with user training, and file conversion. The system may be requiring extensive user training. The initial parameters of the system should be modifies as a result of a programming. A simple operating procedure is provided so that the user can understand the different functions clearly and quickly. The different reports can be obtained either on the inkjet or dot matrix printer, which is available at the disposal of the user. The proposed system is very easy to implement. In general implementation is used to mean the process of converting a new or revised system design into an operational one.

Testing

Testing is the process where the test data is prepared and is used for testing the modules individually and later the validation given for the fields. Then the system testing takes place which makes sure that all components of the system property functions as a unit. The test data should be chosen such that it passed through all possible condition. Actually testing is the state of implementation which aimed at ensuring that the system works accurately and efficiently before the actual operation commence. The following is the description of the testing strategies, which were carried out during the testing period.

System Testing

Testing has become an integral part of any system or project especially in the field of information technology. The importance of testing is a method of justifying, if one is ready to move further, be it to be check if one is capable to with stand the rigors of a particular situation cannot be underplayed and that is why testing before development is so critical. When the software is developed before it is given to user to use the software must be tested whether it is solving the purpose for which it is developed. This testing involves various types through which one can ensure the software is reliable. The program was tested logically and pattern of execution of the program for a set of data are repeated. Thus the code was exhaustively checked for all possible correct data and the outcomes were also checked.

Module Testing

To locate errors, each module is tested individually. This enables us to detect error and correct it without affecting any other modules. Whenever the program is not satisfying the required function, it must be

corrected to get the required result. Thus all the modules are individually tested from bottom up starting with the smallest and lowest modules and proceeding to the next level. Each module in the system is tested separately. For example the job classification module is tested separately. This module is tested with different job and its approximate execution time and the result of the test is compared with the results that are prepared manually. The comparison shows that the results proposed system works efficiently than the existing system. Each module in the system is tested separately. In this system the resource classification and job scheduling modules are tested separately and their corresponding results are obtained which reduces the process waiting time.

Integration Testing

After the module testing, the integration testing is applied. When linking the modules there may be chance for errors to occur, these errors are corrected by using this testing. In this system all modules are connected and tested. The testing results are very correct. Thus the mapping of jobs with resources is done correctly by the system.

Acceptance Testing

When that user find no major problems with its accuracy, the system passers through a final acceptance test. This test confirms that the system needs the original goals, objectives and requirements established during analysis without actual execution which elimination wastage of time and money acceptance tests on the shoulders of users and management, it is finally acceptable and ready for the operation.

Test Case Id	Test Case Name	Test Case Desc.	Test Steps			Test Case Status	Test Priority
			Step	Expected	Actual		
01	upload anti money landering dataset	Verify Anti Moneylandering Dataset updated or not	If antimony Landering dataset is May not be Uploaded	we cannot do any further operations	we can do further operations	High	High
02	Preprosser dataset	Verify preprosser dataset is updated or not	If preprosser Dataset is may not be Updated	we cannot do any further operations	we can do further operations	High	High
03	Run random forest with Transaction	Verify Random forest with Transaction is Run Sucessfully or not	If Random Forest with Transaction is may not be Run Sucessfully	We cannot run operation	We can Run the Operation	High	High

04	Run with Transaction and timefrequency	Verify transaction and time frequency Run successfully or not	If Transaction and Time Frequency is may not run Sucessfully	We cannot run operation	We can Run the Operation	High	High
05	Comparision Graph	Verify Comparision Graph Displayed Sucessfully or not	If Comparision Graph May not Displayed Sucessfully	We cannot Get similar graph	We can get similar Graph	High	High
06	predict money landuring from test data	Verify Money Landering from test data is Predicted or not	If Money Landering from test data not predicted	We cannot generate Money Landering from test data	We can Generate Money Landering from test data	High	High
07	Close application	Verify application closed or not	If application is not closed	We cannot run operation	We can run operation	High	High

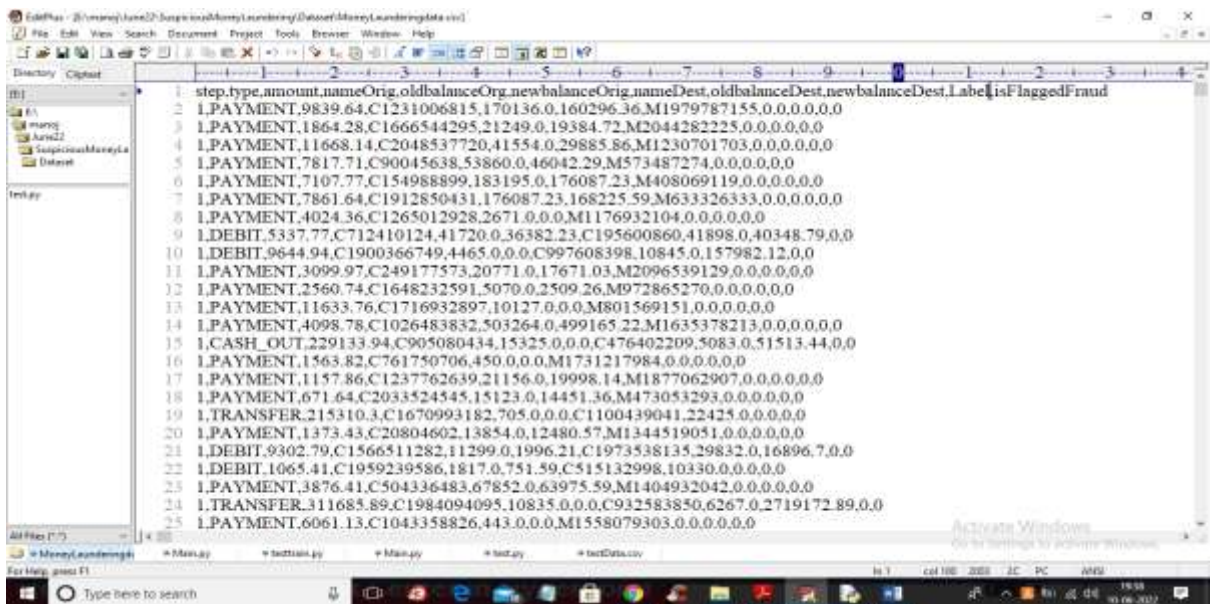
9. RESULTS

Now-a-days due to online transactions it's become easier for criminal to transfer money to one and other to complete criminal task and this money transfer is called as Money-Laundering. To detect such transaction many Anti-Money Laundering technique was used where FINANACIAL BANK UNITS will form suspicious transaction to Financial Intelligence Unit (FIU) and FIU will forward this details to Law Enforcement Agencies (LEA) for further investigation and if transaction found suspicious then they will inform to police who will recover transaction money for public use.

This existing manual transaction investigation success ratio is 5% and failed ratio is 95% and to automate such process many machine learning algorithms was used but they were dependent on transaction data and its accuracy is not up to the mark so author of this paper using Random Forest algorithm and then experiment with Transaction data and Time Frequency data. Random Forest Training with Time Frequency data is giving high FSCORE and HIGH True Positive Rate compare to normal transaction data.

Time Frequency data can be obtained from transaction dataset by applying Fourier Transform called FFT. FFT will return 11 features such as Mean, Variance, Skewness, Kurtosis etc.

To train random forest algorithm we have used TRANSACTION data which contain 11 features and below is the dataset screenshot.



step	type	amount	nameOrig	oldbalanceOrig	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	Label	isFlaggedFraud
1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155	0.0	0.0	0.0	0
2	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225	0.0	0.0	0.0	0
3	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1230701703	0.0	0.0	0.0	0
4	PAYMENT	7817.71	C90045638	53860.0	46042.29	M573487274	0.0	0.0	0.0	0
5	PAYMENT	7107.77	C154988899	183195.0	176087.23	M408069119	0.0	0.0	0.0	0
6	PAYMENT	7861.64	C1912850431	176087.23	168225.59	M633326333	0.0	0.0	0.0	0
7	PAYMENT	4024.36	C1265012928	2671.0	0.0	M1176932104	0.0	0.0	0.0	0
8	DEBIT	5337.77	C712410124	41720.0	36382.23	C195600860	41898.0	40348.79	0.0	0
9	DEBIT	9644.94	C1900366749	4465.0	0.0	C997608398	10845.0	157982.12	0.0	0
10	PAYMENT	3099.97	C249177573	20771.0	17671.03	M2096539129	0.0	0.0	0.0	0
11	PAYMENT	2560.74	C1648232591	5070.0	2509.26	M972865270	0.0	0.0	0.0	0
12	PAYMENT	11633.76	C1716932897	10127.0	0.0	M801569151	0.0	0.0	0.0	0
13	PAYMENT	4098.78	C1026483832	503264.0	499165.22	M1635378213	0.0	0.0	0.0	0
14	CASH_OUT	229133.94	C905080434	15325.0	0.0	C476402209	5083.0	51513.44	0.0	0
15	PAYMENT	1563.82	C761750706	450.0	0.0	M1731217984	0.0	0.0	0.0	0
16	PAYMENT	1157.86	C1237762639	21156.0	19998.14	M1877062907	0.0	0.0	0.0	0
17	PAYMENT	671.64	C2033524545	15123.0	14451.36	M473053293	0.0	0.0	0.0	0
18	TRANSFER	215310.3	C1670993182	705.0	0.0	C1100439041	22425.0	0.0	0.0	0
19	PAYMENT	1373.43	C20804602	13854.0	12480.57	M1344519051	0.0	0.0	0.0	0
20	DEBIT	9302.79	C1566511282	11299.0	1996.21	C1973538135	29832.0	16896.7	0.0	0
21	DEBIT	1065.41	C1959239586	1817.0	751.59	C515132998	10330.0	0.0	0.0	0
22	PAYMENT	3876.41	C504336483	67852.0	63975.59	M1404932042	0.0	0.0	0.0	0
23	TRANSFER	311685.89	C1984094095	10835.0	0.0	C932583850	6267.0	2719172.89	0.0	0
24	PAYMENT	6061.13	C1043358826	443.0	0.0	M1558079303	0.0	0.0	0.0	0

Fig 17: Transaction data features

In above dataset first row contains dataset column names and remaining rows contains dataset values.

SCREENSHOTS

To run project double click on 'run.bat' file to get below screen

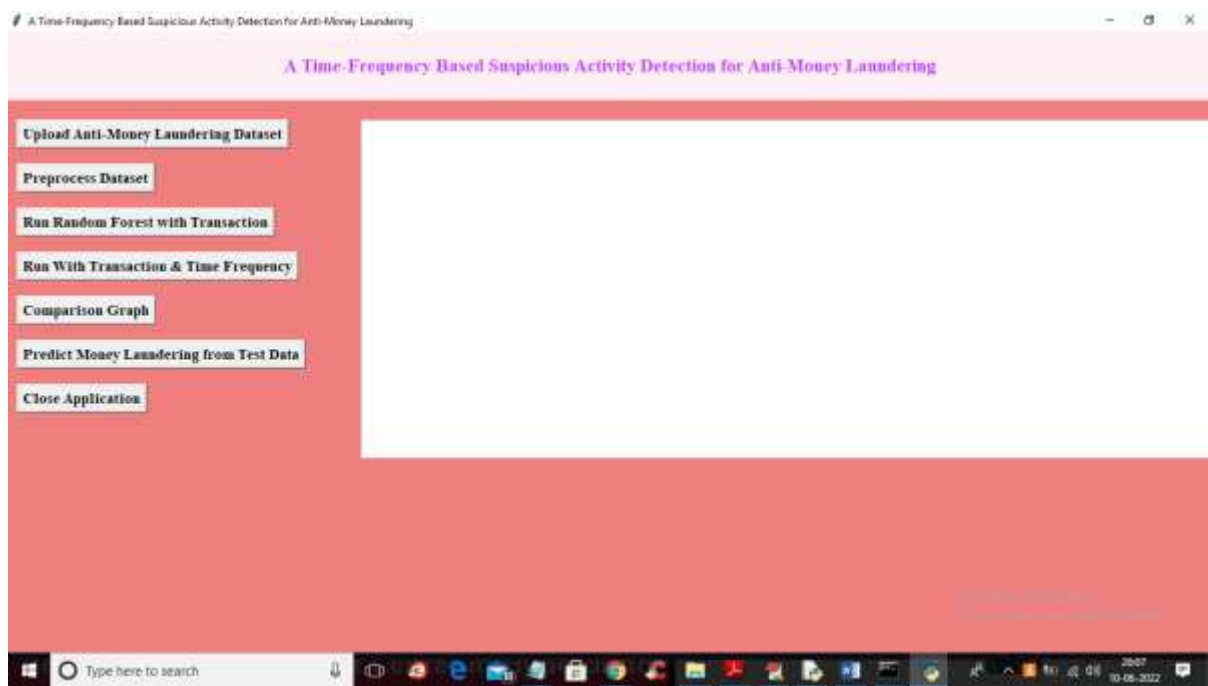


Fig 18: User Interface

In above screen click on 'Upload Anti-Money Laundering Dataset' button to upload dataset and get output.

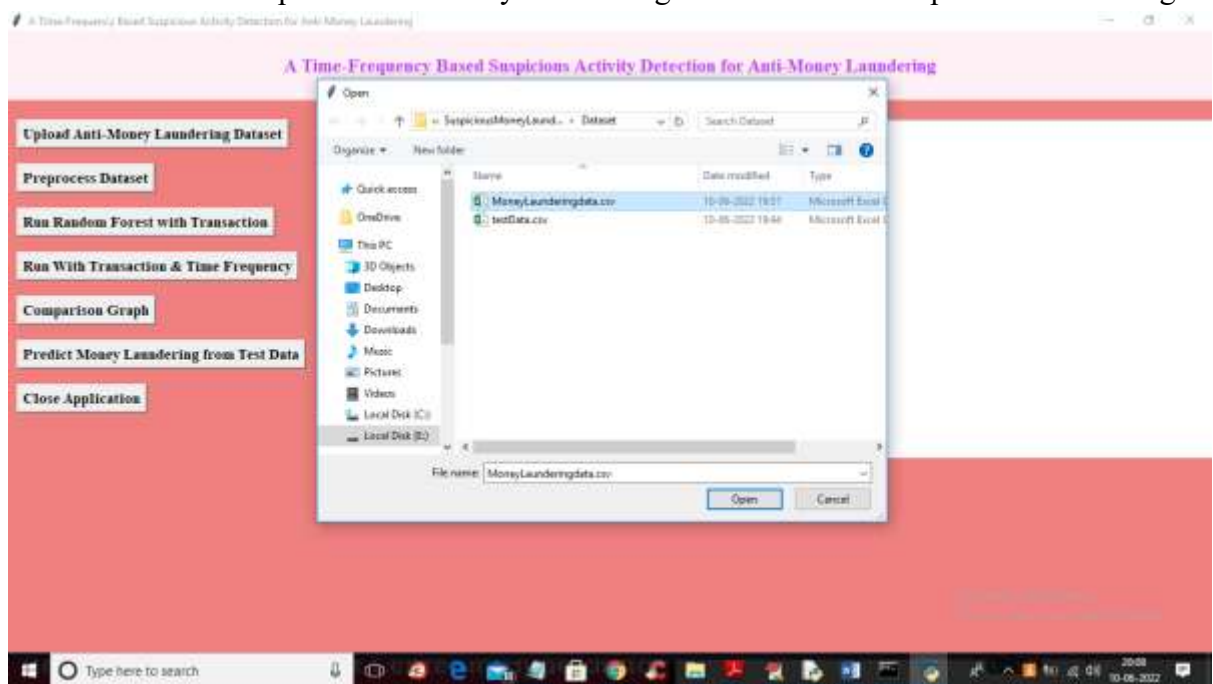


Fig 19: Upload AML dataset

In above screen selecting and uploading 'Money Laundering' dataset and then click on 'Open' button to upload dataset and get below output.



Fig 20: Graph

In above screen we can see dataset loaded and dataset contains some numeric and non-numeric data so we need to Preprocess data to convert into numeric and in above graph x-axis represents labels where 0 means Normal transaction and 1 means Money-Laundering transaction and y-axis represents counts. Now close above graph and then click on 'Preprocess Dataset' button to clean dataset and get below output.

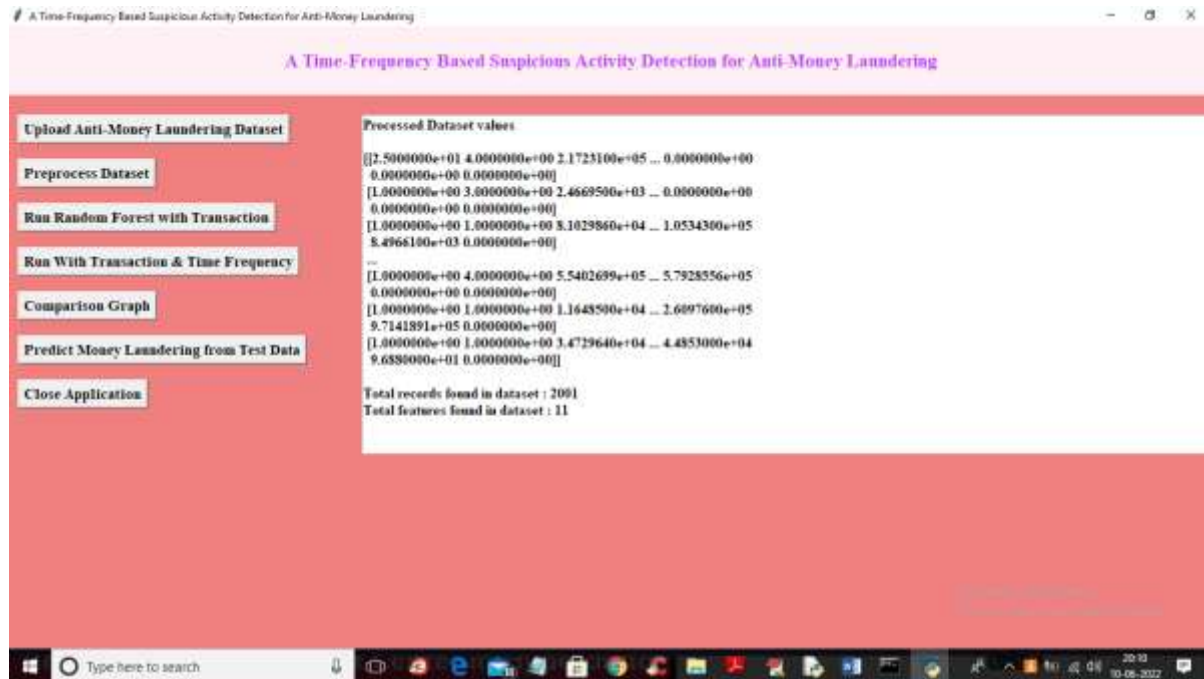


Fig 21: Processed Dataset Values

In above screen we can see entire dataset converted to numeric format and in last two lines we can see dataset contains 2001 records and each record contains 11 features and now dataset is ready and now click on 'Run Random Forest with Transaction' button to train Random Forest on transaction dataset and get below output.

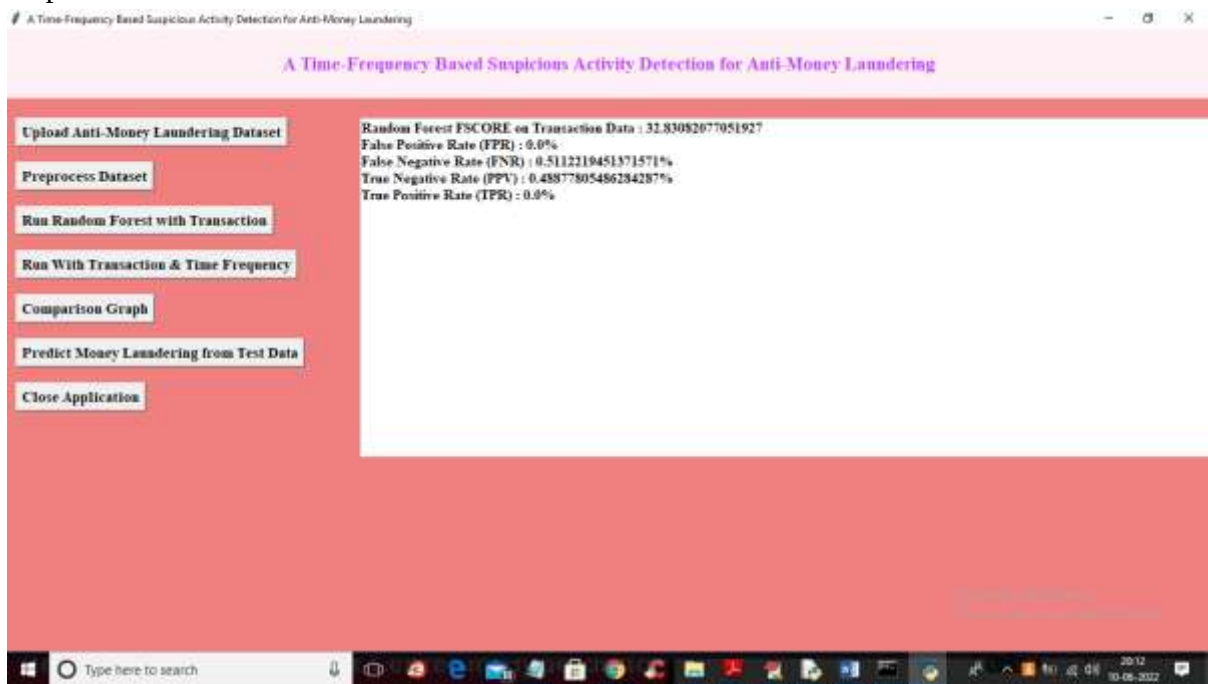


Fig 22: Random Forest F-Score on Transaction Data

In above screen with Random Forest on transaction data we got FSCORE as 32% and True Positive Rate (TPR) as 0. So transaction data features will not allow Random Forest to train perfectly and now click on 'Run With Transaction & Time Frequency' button to convert Transaction data into Time Frequency and then retrain Random Forest to get below output.

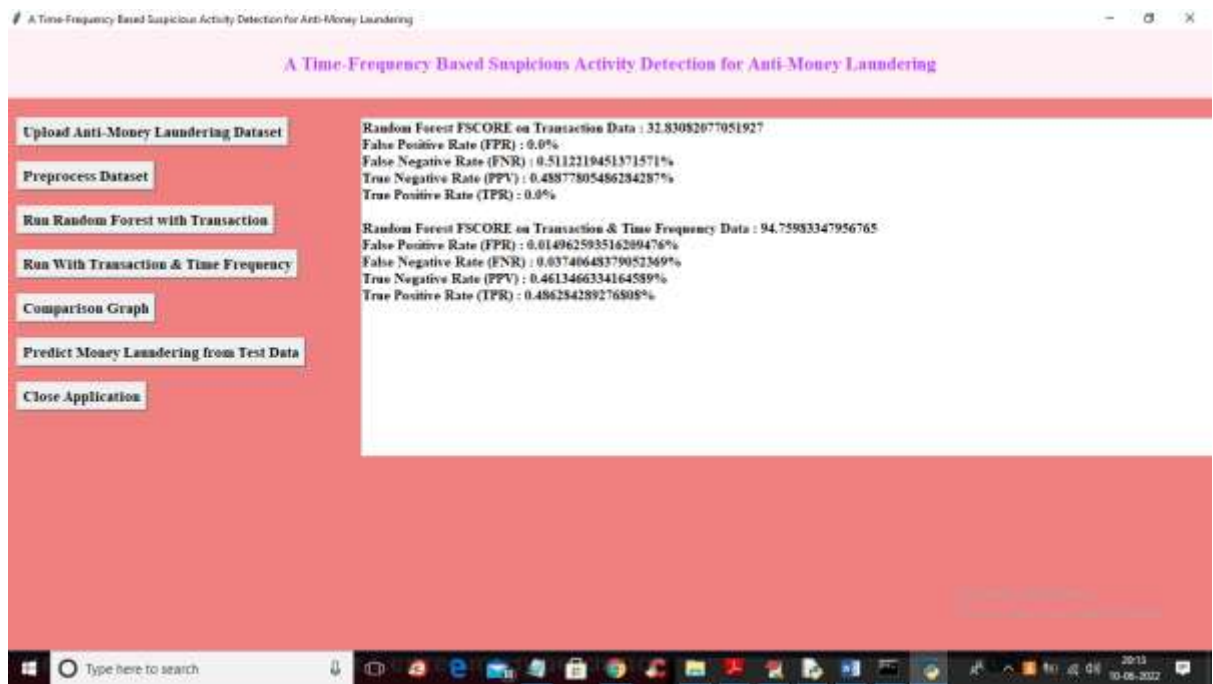


Fig 23: Random Forest F-Score on Transaction & Time Frequency Data

In above screen Random Forest with Time Frequency Transaction data got 94% FSCORE and TPR as 45% so by converting transaction data into time frequency we can build accurate prediction model. Now click on 'Comparison Graph' button to get below graph

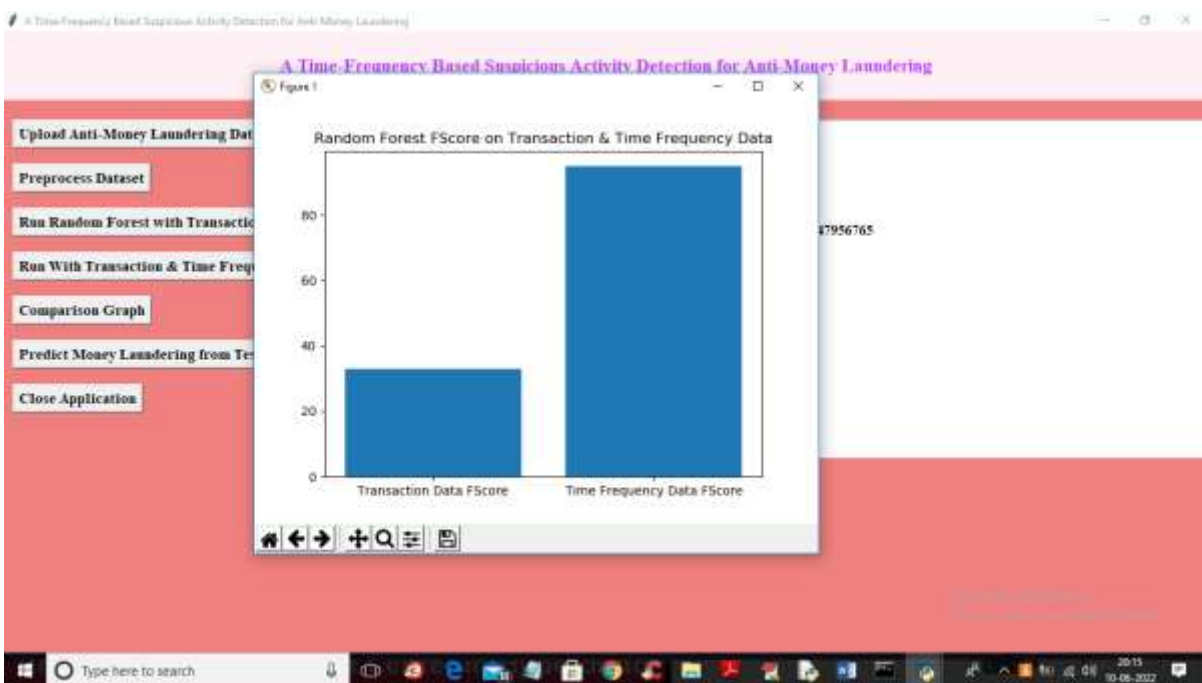


Fig 24: Comparison Graph

In above graph x-axis represents technique name and y-axis represents FSCORE and in both techniques Time Frequency got high FSCORE and now close above graph and then click on 'Predict Money

Laundrying from Test Data' button to upload test data and get prediction output.

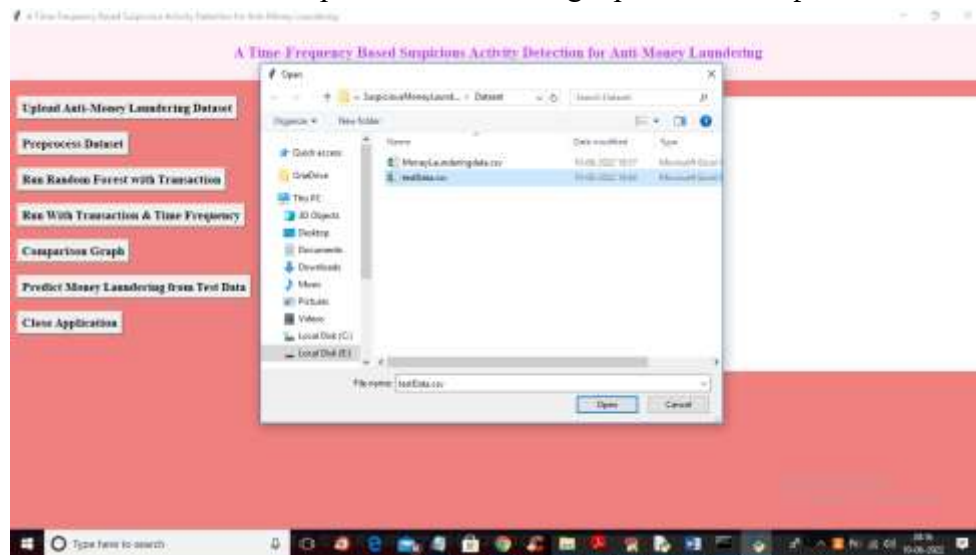


Fig 25: Test Dataset

In above screen selecting and uploading 'testData.csv' file and then click on 'Open' button to get below output

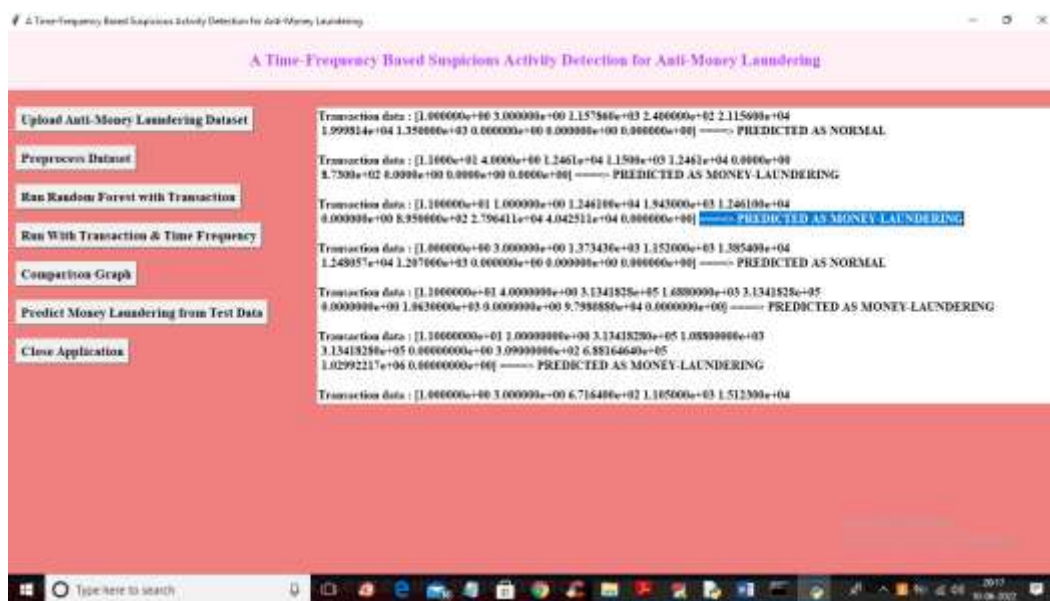


Fig 26: Prediction

In above screen in square bracket we can see transaction test data and after arrow symbol $\Rightarrow$ we got prediction output as 'Normal or Money Laundering'

10. CONCLUSION

In this paper, we have shown that adding time-frequency features, simplifies the feature selection process and improves the quality of the data science model. Time-frequency features such as mean, variance, Kurtosis, and skewness have been used for the first time in machine learning model training for suspicious transaction detection. Therefore, the feature engineering stage can be shortened by calculating the proposed time-frequency feature set. This potentially saves many person-months of modeling studies for the financial institutions.

The proposed solution has been implemented in Python and the high-level of accuracy has been proven on real financial data. The generalized solution can easily be adapted to detect suspicious transactions in various organizations. An analysis of actual customer data indicates that time-frequency features can distinguish between suspicious and clear cases, improving AUC and the efficiency of the transaction monitoring system. Among different time-frequency characteristics, Kurtosis provided the maximum differentiation in the model. The gains in accuracy and the capability of detecting money laundering cases that were not detectable before can save financial institutions from regularity fines and HR cost in the order of millions of USD. In this work, only a low complexity Fourier transform based approach is utilized for frequency domain analysis.

As a future work, the time-frequency analysis can be accomplished with other types of linear and non-linear transforms. There are also potential gains in comparing multiple window lengths, increment sizes and making the analysis in multiple banking channels (such as ATM, Branch, Web). Also, the same analysis can be extended to investigate the characteristics of networks rather than single entities.

In particular, when a customer has multiple accounts in multiple banks, the whole picture can only be analyzed by the FIUs. Therefore, repeating this study with additional FIU data would be beneficial as well.

Overall, time-frequency features offer substantial potential for financial behavior analysis. This study establishes a strong foundation for next-generation AML systems, where enhanced detection accuracy, reduced modeling effort, and improved operational efficiency can together transform the way institutions combat financial crime.

11. FUTURE ENHANCEMENTS

The proposed time-frequency based framework has demonstrated significant improvement in detecting suspicious financial transactions. However, several enhancements can be explored in future research to further improve the effectiveness, scalability, and practical deployment of the system.

1. Integration of Advanced Time–Frequency Techniques

The current system utilizes the **Fast Fourier Transform (FFT)** for converting transaction data into the frequency domain. Future research can explore more advanced time–frequency analysis techniques such as Wavelet Transform (WT) and Short-Time Fourier Transform (STFT).

2. Incorporation of Deep Learning Models

The proposed framework uses a **Random Forest classifier** for suspicious activity detection. Future work can extend the system by integrating deep learning architectures such as **Convolutional Neural Networks (CNN)**, **Recurrent Neural Networks (RNN)**, and **Long Short-Term Memory (LSTM)** models.

3. Evaluation on Large-Scale Multi-Bank Datasets

The current experiment was conducted on a dataset with a limited number of transaction records. Future research can evaluate the proposed framework using **large-scale datasets collected from multiple financial institutions**. This would help improve model generalization, robustness, and scalability for real-world banking environments.

4. Graph-Based Transaction Network Analysis

Money laundering activities often involve networks of interconnected accounts rather than isolated transactions. Future enhancements can incorporate **graph-based analysis techniques** to model relationships between accounts and identify suspicious transaction networks.

5. Real-Time Fraud Detection System

Another potential enhancement is the development of a **real-time transaction monitoring system** that integrates the proposed framework into banking infrastructure. Real-time analysis can enable financial institutions to detect suspicious transactions instantly and reduce financial risks.

6. Automated Alert Prioritization System

Future systems can include an **intelligent alert prioritization mechanism** that ranks suspicious transactions based on risk levels. This would help compliance officers focus on high-risk cases and reduce the workload caused by large volumes of alerts.

12. REFERENCE

- [1] Z. Chen, Y. Li, and H. Zhang, “Graph neural network based detection of money laundering activities in 9 financial transaction networks,” *IEEE Access*, vol. 10, pp. 84521–84533, 2022.
- [2] A. Kumar, P. Singh, and R. Sharma, “Ensemble machine learning techniques for financial fraud detection in banking systems,” *IEEE Transactions on Computational Social Systems*, vol. 10, no. 2, pp. 420–431, Apr. 2023.
- [3] M. Rahman, K. Ahmed, and S. Islam, “Isolation forest based anomaly detection for anti-money laundering transaction monitoring,” *IEEE Access*, vol. 11, pp. 56712–56724, 2023.
- [4] J. Almeida, P. Costa, and L. Rodrigues, “Deep learning approaches for financial fraud detection using long short-term memory networks,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 4, pp. 5120–5131, Apr. 2024.
- [5] A. Gupta and V. Sharma, “Time series analysis of financial transactions for suspicious activity detection,” *IEEE Access*, vol. 12, pp. 23874–23886, 2024.
- [6] T. Patel and R. Shah, “Feature engineering techniques for machine learning based anti-money laundering systems,” in *Proc. IEEE Int. Conf. Big Data*, Washington, DC, USA, 2024, pp. 4123–4130.
- [7] H. Zhao, X. Wang, and J. Liu, “Hybrid deep learning framework for detecting complex financial fraud patterns,” *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 1, pp. 98–109, Jan. 2025.
- [8] K. Singh, D. Verma, and M. Gupta, “Advanced anomaly detection for anti-money laundering using hybrid machine learning techniques,” *IEEE Access*, vol. 13, pp. 12450–12463, 2025.
- [9] L. Zhang, Q. Sun, and Y. Wang, “Financial transaction anomaly detection using autoencoder-based deep learning models,” *IEEE Access*, vol. 10, pp. 105214–105226, 2022.
- [10] P. R. Kumar and S. Gupta, “Comparative analysis of machine learning algorithms for fraud detection in banking systems,” in *Proc. IEEE Int. Conf. Data Mining Workshops (ICDMW)*, Orlando, FL, USA, 2022, pp. 675–682.
- [11] M. Ahmed and N. Mahmood, “A survey of machine learning approaches for financial fraud detection,” *IEEE Access*, vol. 11, pp. 27840–27855, 2023.
- [12] D. Li, X. Zhou, and Y. Chen, “Graph-based analysis for detecting suspicious transaction patterns in financial networks,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 1, pp. 212–224, Jan. 2024.
- [13] R. Das, S. Banerjee, and P. Bhattacharya, “Hybrid anomaly detection framework for financial transaction monitoring,” *IEEE Access*, vol. 12, pp. 45678–45690, 2024.
- [14] S. K. Patel and A. Mehta, “Machine learning driven anti-money laundering detection using behavioral transaction analysis,” in *Proc. IEEE Int. Conf. Artificial Intelligence and Data Analytics*, Singapore, 2025, pp. 210.
- [15] Y. Liu, J. Huang, and X. Zhao, “Deep representation learning for financial fraud detection in large-scale transaction datasets,” *IEEE Transactions on Big Data*, vol. 11, no. 2, pp. 340–352, 2025.